

Python Learning Materials

Complete Compilation — Day 1 to Day 20

All 20 source PDFs are compiled below in day order. Duplicate copies from the uploaded materials were not repeated.

Day	Topic
1	Introduction
2	Data Types
3	Operators
4	Control Statements
5	Control Statements & Functions
6	Special Functions
7	Modules & Data Structure
8	Lists & Tuples
9	Arrays & Strings
10	Sets & Dictionaries
11	File Handling
12	Exceptions
13	Class, Object & Constructor
14	Inheritance
15	Polymorphism, Operators & Attributes
16	Multithreading
17	GUI Programming
18	GUI Programming
19	GUI Programming
20	Database

INTRODUCTION TO PYTHON

History of Python:

- Ø Python was named after the person called Monty Python.
- Ø Python was developed by **Guido van Rossum** between 1985 - 1990 at the National Research Institute for Mathematics and Computer Science in Netherlands.
- Ø Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, Smalltalk, Unix shell, and other scripting languages.
- Ø Python is a cross platform programming language, i.e., it is capable to run on multiple platforms like windows, Mac OS X, Linux, Unix.
- Ø Python is one of the free and open source software.
- Ø Python is a language which acts as an **interpreter**.
- Ø Python is a high level programming language.

Features of Python:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax.
- **Easy-to-read:** Python code is more clearly defined.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** Low level modules can be added, which makes the customer to add or customize the tools more effectively.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries, and windows systems.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.
- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types, supports dynamic type Checking and also supports automatic garbage collection.

Python is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Methods to start the python interpreter:

There are two types of shell in python

- Ø Immediate mode / Interactive mode.
- Ø Scripting mode.
- Ø Command line execution.

Immediate mode / Interactive mode:

Typing python in the command line will invoke the interpreter in immediate or an interactive mode. We can directly type in python expression and give on enter to obtain the output.

>>> (called as python prompt)

Example:

```
>>> 1+1
2
>>>
```

Scripting mode:

It is helpful in executing the complete python source code. Script programs can be saved for further reference whereas the interactive mode programs cannot be used for further reference.

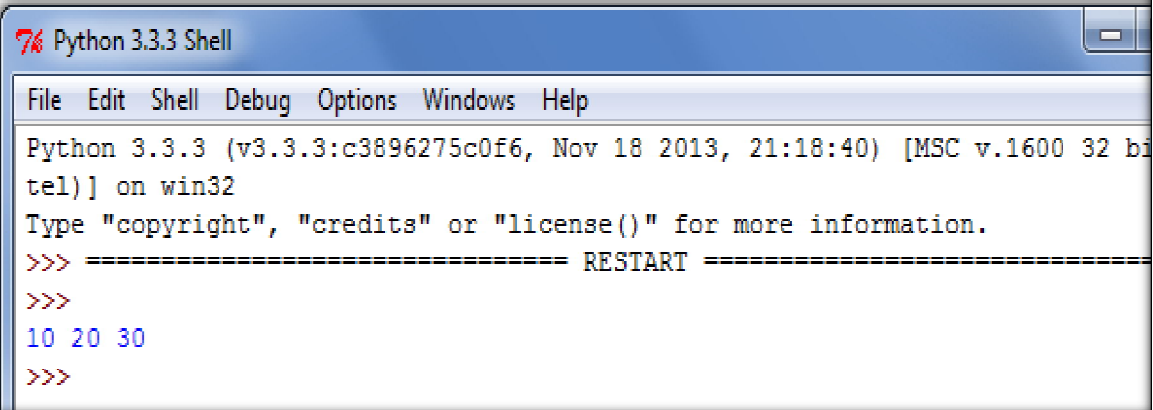
Program need to be saved with the extension called .py

Example:

Save the program in the name called sample.py

Example:

```
a=10
b=20
c=a+b
print(a,b,c)
```



Command line execution:

The command line execution is similar to that of interactive mode. Here the programs cannot be referred further. It provides a simple executing statement of the program.

Example:

```
Python 3.3.3 Shell
Type "help", "copyright", "credits" or "license()" for more information.
>>> name="mouse"
>>> rate=20
>>> qty=5
>>> print(name,rate*qty)
mouse 100
>>> _
```

Steps to open and run the script programs:

- Ø Open the idle(GUI) by clicking STARTà programsà python 3.3à IDLE(GUI).
- Ø Click on Fileà New Fileà Type your programà Save it in your folder by giving any program_name (sample.py)
- Ø To run the program click on F5 (or) click on runà run module.

NOTE:

The program will run or display the output in the immediate mode or interactive mode only, it cannot display in scripting mode.

PYTHON IDENTIFIERS:

- Ø A Python identifier is a name used to identify a variable, function, class, module, or other object.
- Ø An identifier starts with a letter A to Z or a to z, or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).
- Ø Python does not allow punctuation characters such as @, \$, and % within identifiers.
- Ø Python is a case sensitive programming language. Thus, **Manpower** and **manpower** are two different identifiers in Python.

Note:

- For simple programs there is no necessity to declare an identifier.
- No identifier can begin with a number, and no symbols other than underscore are allowed.

Here are naming conventions for Python identifiers:

Ø Class names start with an uppercase letter. All other identifiers start with a lowercase letter.

Ø Starting an identifier with a single leading underscore indicates that the identifier is private.

Ø Starting an identifier with two leading underscores indicates a strongly private identifier.

Ø If the identifier also ends with two trailing underscores, the identifier is a language-defined special name.

Working with identifiers:

A variable can be used randomly at any required place, there is no necessity to declare it earlier as specified in c language. And a variable can be deleted at any non-required position.

Example 1:

```
tel>1 on win32
Type "help", "copyright", "credits" or "
>>> x=4
>>> y="THIS IS A STRING"
>>> x
4
>>> y
'THIS IS A STRING'
>>>
```

Example 2:(overriding a variable)

```
tel>I on win32
Type "help", "copyright", "credits" or "license()"
>>> A= 12345
>>> A
12345
>>> A="CSC COMPUTER EDUCATION"
>>> A
'CSC COMPUTER EDUCATION'
>>> _
```

Example 3:(Deleting a variable)

```
Type "help", "copyright", "credits" or "license()"
>>> value=1000
>>> print(value)
1000
>>> amount=120000
>>> print(amount)
120000
>>> del value
>>> print(value)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'value' is not defined
>>>
```

PYTHON KEYWORDS:

Python consist of about 28 keywords. The keywords are and, assert, except, in, is, lambda, not, elif, else if, import, raise, return, try, while, global, pass, print, continue, def, del, or, break((((((((, exec, finally, for, from.

PYTHON STATEMENTS:

Instructions that a python interpreter can execute are called as python statements. Python statements are classified as

- Ø Single line statements
- Ø Multi line statements
- Ø Multiple statements in single line

Single line statements:

Example1:

```
tel>] on win32
Type 'help', 'copyright', 'credits' or
>>> value1=100
>>> value2=200
>>> result=value1+value2
>>> print(result)
300
>>>
```

Multi line statements:

Example 2:

```
Type 'help', 'copyright', 'credits' or
>>> a=1+2+3+\
... 4+5+6+\
... 7+8+9
>>> print(a)
45
>>>

Type 'help', 'copyright', 'credits' or
>>> number=(1+2+3+
... 4+5+6+7+8+9+10)
>>> print(number)
55
>>>
```

Multiple statement in a single line:

Example 3:

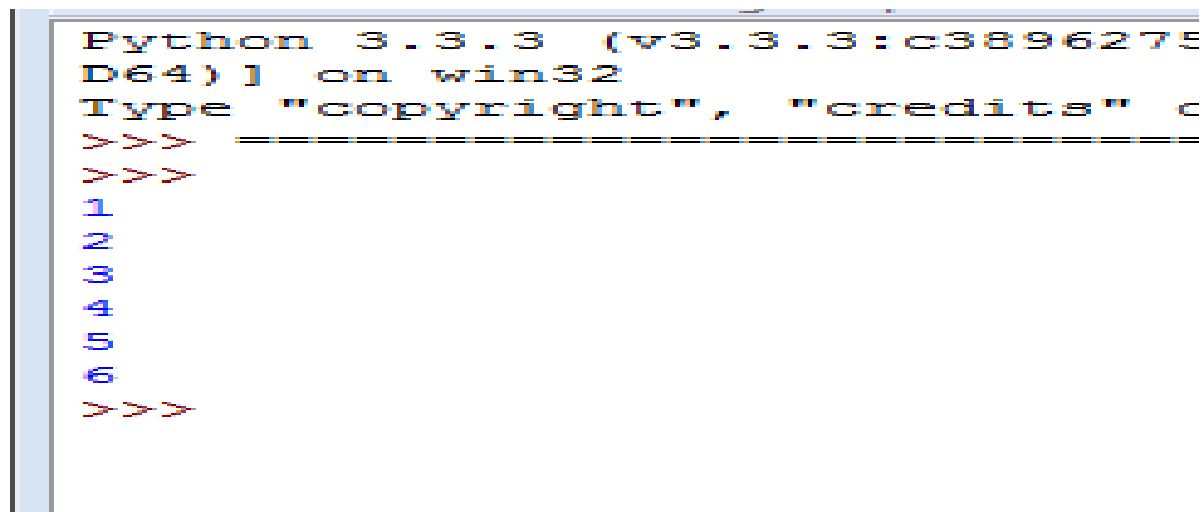
```
Type 'copyright', 'credits' or 'license()' to
>>> m="csc "; n="computer "; o="education"
>>> x,y,z=1000,2000,3000
>>> print(m,n,o)
csc computer education
>>> print(x,y,z)
1000 2000 3000
>>>
```

PYTHON INDENTATION:

Most of the programming languages like C,C++ and some of the others uses braces like {} to define a block of statements, whereas python uses the concept of indentation .

Example 1:

```
for i in range(1,10):  
    print (i)  
    if i==6:  
        break
```



```
Python 3.3.3 (v3.3.3:c3896275  
D64) ] on win32  
Type "copyright", "credits" c  
>>> =====  
>>>  
1  
2  
3  
4  
5  
6  
>>>
```

PYTHON COMMENTS:

Single line comment:

Example:

this indicates the single line comment

Multi line comment:

Example:

```
""" This is  
the example of  
multi line comment"""
```

(or)

```
'''Multiline comment  
Can also be represented  
in this format'''
```


What can you do with python:

Python is used to create web and desktop applications, and some of the most popular web applications like Instagram, YouTube, Spotify all has been developed in Python, and you can also develop next big thing by using Python.

SAMPLE PROGRAMS:

Program1:

```
#sample program  
print ("Hello, World!")
```

Program2:

```
#program to add two numbers in python  
a = b = 20 #declare two variables and store 20 into them  
print (a+b) #final output
```

Program3:

```
#program to display a value, override it and to delete it from its memory  
a=10          #single line statement  
b= 10.12+10.13+10.14+\  
    10.15+10.16          #multi line statement  
c,d,e,f=(1,'keyboard',"cpu",1234.567)      #multiple statement in a single line  
print(a)  
print(b)  
print(c,d,e,f)  
del (a)       #deleting a variable from its memory  
print("A value has been deleted from its memory")  
print(a)
```

PYTHON DATATYPES:

Standard Data Types:

The data stored in memory can be of many types. For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.

Python has five standard data types:

- Ø Numbers
- Ø String
- Ø List
- Ø Tuple
- Ø Dictionary

Python Numbers:

Number datatypes store numeric values. Number objects are created when you assign a value to them.

Python supports four different numerical types:

- Ø int (signed integers)
- Ø long (long integers, they can also be represented in octal and hexadecimal)
- Ø float (floating point real values)
- Ø complex (complex numbers)

Type Conversion(casting):

Python has the capability and feature to convert within an expression containing the mixture of different types of values internally.

- int(v): is used to convert any value 'v' to a plain integer
- long(v): is used convert a value 'v' to a long integer
- float(v): is used convert a value 'v' to floating point value
- complex(v): is used convert a value 'v' to the complex number having real part 'v' and imaginary part as 0
- complex (u ,v): is used convert values u and v to the complex number with real part 'u' and imaginary part 'v'.

Example:

```
num1=10.1234
num2=100.98
print (int (num1)+num2)
```

Python Strings

- ü Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes.
- ü The plus (+) sign is the string concatenation operator and the asterisk (*) is the repetition operator.
- ü Subsets of strings can be taken using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.
- ü The type() function is used to know which class a variable or a value belongs to.

Input:

```
s1="Python "  
s2="is considered as "  
s3="one of the "  
s4="interesting languages!"  
print("Given string is ", s1+s2+s3+s4)  
print('Graphical Animation' * 3)  
print(s1[0])  
print(s4[0:3])  
print(s1[1:])  
print(s4[:3])
```

Output:

```
>>>  
Given string is  Python is considered as one of the interesting languages!  
Graphical AnimationGraphical AnimationGraphical Animation  
P  
int  
ython  
int  
>>>
```

Python Lists:

- ü List is considered as ordered sequence of items.
- ü A list contains items separated by commas and enclosed within square brackets ([]).
- ü To some extent, lists are similar to arrays in C.
- ü One difference between them is that all the items belonging to a list can be of different data type.
- ü The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end -1.

Input:

```
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tinylist = [123, 'john']
print (list) # Prints complete list
print (list[0]) # Prints first element of the list
print (list[1:3]) # Prints elements starting from 2nd till 3rd
print (list[2:]) # Prints elements starting from 3rd element
print (tinylist * 2) # Prints list two times
print (list + tinylist) # Prints concatenated lists
```

Output:

```
['abcd', 786, 2.23, 'john', 70.2]
abcd
[786, 2.23]
[2.23, 'john', 70.2]
[123, 'john', 123, 'john']
['abcd', 786, 2.23, 'john', 70.2, 123, 'john']
>>>
```

Python Tuple:

- ü A tuple is another ordered sequence of datatype that is similar to the list. A tuple consists of a number of values separated by commas.
- ü Unlike lists, tuples are enclosed within parentheses.
- ü The main differences between lists and tuples are:
 - Lists are enclosed in brackets ([]) and their elements and size can be changed, while tuples are enclosed in parentheses (()) and cannot be updated.
- ü Tuples acts as a read only format.

Input:

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
tinytuple = (123, 'john')
print (tuple) # Prints complete list
print (tuple[1:3]) # Prints elements starting from 2nd till 3rd
print (tuple[2:]) # Prints elements starting from 3rd element
print (tinytuple * 2) # Prints list two times
print (tuple + tinytuple) # Prints concatenated lists
```

Output:

```
('abcd', 786, 2.23, 'john', 70.2)
(786, 2.23)
(2.23, 'john', 70.2)
(123, 'john', 123, 'john')
('abcd', 786, 2.23, 'john', 70.2, 123, 'john')
>>>
```

Difference between a list and tuple:

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tuple[2] = 1000 # Invalid syntax with tuple
list[2] = 1000 # Valid syntax with list
```

Python Dictionary:

- ü Python's dictionaries are kind of hash table type.
- ü They work like associative arrays or hashes found in Perl and consist of unordered collections of key-value pairs.
- ü A dictionary key can be almost any Python type, but are usually numbers or strings.
- ü Values, on the other hand, can be any arbitrary Python object.
- ü Dictionaries are enclosed by curly braces ({ }) and values can be assigned and accessed using square braces ([]).

Input:

```
dict = {}
dict['one'] = "This is one"
dict[2] = "This is two"
tinydict = {'name': 'john', 'code':6734, 'dept': 'sales'}
print (dict['one']) # Prints value for 'one' key
print (dict[2]) # Prints value for 2 key
print (tinydict) # Prints complete dictionary
print (tinydict.keys()) # Prints all the keys
print (tinydict.values()) # Prints all the values
```

Output:

```
>>>
This is one
This is two
{'code': 6734, 'dept': 'sales', 'name': 'john'}
dict_keys(['code', 'dept', 'name'])
dict_values([6734, 'sales', 'john'])
>>>
```

Python Set:

- ü A set is a collection which is unordered and unindexed. In Python sets are written with curly brackets.

Input:

```
thisset = {"apple", "banana"}
print(thisset)
```

ESCAPE CHARACTERS:

These are special characters represented by a backslash followed by the character(s), and they are used for special purposes. They can be interpreted using both single and double quote. The lists of Escape Characters in Python are:

- \a: alert
- \b: backspace
- \cx: ControlX
- \e: escape
- \f: Form feed
- \n: New line or next line
- \r: carriage return
- \s: space
- \t: tab
- \v: Vertical Tab

Sample Programs On Datatypes

Program1:(integer)

```
x = 10.5
y = 5
#without type cast
print (x + y)
#after type cast
print (int(x) + y)
```

Program2:(string)

```
str1 = 'Hello Python'
str2 = "String Chapter"
print ("First value is: ", str1)
print ("Second value is: ", str2)
print ("First sub-string in str1 is: ", str1[0])
print ("Set of sub-string in str2 is: ", str2[2:6])
print ("Updated string will be: ", str1[:11]+ "Python")
print ("The appended string is: ", str1 + str2)
print ("Repitition on the string is: ", str1[0:5] *3)
print(len(str1))
```

```
print(str1.lower())
print(str1.upper())
print(str1.replace("H", "J"))
```

Program3:(list)

```
a = ['computersc', 'IT', 'CSE'];
b = [1993, 2016];
c = [2, 4, 6, "g", "k", "s"];
d = list(("apple", "banana", "cherry"))
print(a+b+c+d)
print(a *3)
print ("a[0]", a[0])
print ("Updated value of 'b' index is: ", b)
print ("c[2:4]", c[2:4])
print ("Second value of the 'a' list is : ",a[1])
a[1] = 'Robotics'
del b;
print("The contents of the list 'b' is deleted")
print(d)
d.append("damson")
print(d)
d.remove("banana")
print(d)
print(len(d))
```

Program4:(tuple)

```
tupl1 = ('Microsoft Word', 'Microsoft Excel', 'Microsoft Powerpoint');
tupl2 = (1990, 1996);
tupl3 = (2, 4, 6, 8, 10, 12, 14, 16);
print ("tupl1[0]", tupl1[0])
print("tupl2 is",tupl2)
print ("tupl3[2:4]", tupl3[2:4])
tupl3 = tupl1 + tupl2
print (tupl3)
print("Length of Tuple 3 is ",len(tupl3))
del tupl2;
print(tupl2)
```

Program5:(dictionary)

```
a = {'Bookname' : 'Python', 'Price' : 210}
print(a)
#Adding new entries
```



```

a ['Author'] = 'TutorialsCloud' ;
a ['Discount']= '10 Percent';
print(a)
#Updating an Entry
a ['Price'] = 200;
a = {'Author': 'Cloud'}
print("The new entry and updated value of the dictionary 'a' is ",a)
a.clear();#removes all entries in dictionary & makes the dictionary empty
del a#it Deletes the entire dictionary
a1= {
    "Word": "Document",
    "Excel": "Book",
    "Powerpoint": "Slide"
}
print(a1['Word'])
print(a1)

```

Program6:(set)

```

set1 = {"Notepad", "Wordpad"}
print(set1)
set1.add("MS Paint")
print(set1)
set1.remove("Wordpad")
print(set1)
print("Length of the set is ",len(set1))
print(set1[0]) #-----error

```

PYTHON OPERATORS:

Operators are the constructs which can manipulate the value of operands. Python divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

Arithmetic operators:

Assume variable a holds 10 and variable b holds 20, then:

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators.	$a ** b = 10$ to the power 20
//Floor Division	The division of operands where the result is the quotient in which the digits after the decimal point are removed	$9 // 2 = 4$ and $9.0 // 2.0 = 4.0$

Assignment operators:

Assume variable a holds 10 and variable b holds 20, then:

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-=	It subtracts right operand from the	$c -= a$ is equivalent

Subtract AND	left operand and assign the result to left operand.	to $c = c - a$
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand.	$c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand.	$c /= a$ is equivalent to $c = c / a$
%= Modulus AND	It takes modulus using two operands and assign the result to left operand	$c \% = a$ is equivalent to $c = c \% a$
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand.	$c ** = a$ is equivalent to $c = c ** a$
//= Floor Division	It performs floor division on operators and assign value to the left operand.	$c //= a$ is equivalent to $c = c // a$

Comparison operator:

These operators compare the values on either sides of them and decide the relation among them. They are also called Relational operators.

Assume variable a holds 10 and variable b holds 20, then:

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	$(a == b)$ is not true.
!=	If values of two operands are not equal, then condition becomes true.	$(a != b)$ is true.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	$(a > b)$ is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.	$(a < b)$ is true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	$(a >= b)$ is not true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	$(a <= b)$ is true.

Python Logical Operators:

Assume variable a holds 10 and variable b holds 20 then:

Operators	Description	Example
Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
Logical NOT	Used to reverse the logical state of its operand.	Not (a and b) is false.

Python Identity Operators:

Identity operators compare the memory locations of two objects.

Operators	Description	Example
Is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise	x is not y, here is not results in 1 if id(x) is not equal to id(y).

Python Membership Operators

Python's membership operators test for membership in a sequence, such as strings, lists, or tuples.

Operators	Description	Example
In	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
Not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Python Bitwise Operators

Bitwise operator works on bits and performs bit by bit operation. Assume if a = 60; and b = 13; Now in binary format they will be as follows:

a = 0011 1100

b = 0000 1101

a&b = 0000 1100

a|b = 0011 1101

a^b = 0011 0001

~a = 1100 0011

Operators	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands.	(a & b) = 12 (means 0000 1100)
 Binary OR	It copies a bit if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the Right.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand	a >> 2 = 15 (means 0000 1111)

Sample programs:

Program 1: (Arithmetic operators)

```

a = 5
b = 4
c = 0
c = a + b
print ("Line 1 - Value of c is ", c)
c = a - b
print ("Line 2 - Value of c is ", c)
c = a * b
print ("Line 3 - Value of c is ", c)
c = a / b
print ("Line 4 - Value of c is ", c)
c = a % b
print ("Line 5 - Value of c is ", c)
c = a**b
print ("Line 6 - Value of c is ", c)
c=a//b
print ("Line 7 - Value of c is ", c)

```

Program 2: (Assignment operators)

```
a = 10
c = 5
c += a
print ("Line 1 - Value of c is ", c)
c -= a
print ("Line 2 - Value of c is ", c)
c *= a
print ("Line 3 - Value of c is ", c)
c /= a
print ("Line 4 - Value of c is ", c)
c = 2
c %= a
print ("Line 5 - Value of c is ", c)
c **= a
print ("Line 6 - Value of c is ", c)
c //= a
print ("Line 7 - Value of c is ", c)
```

Program 3: (Comparison operators)

```
a = 757
b = 789
if ( a == b ):
    print ("Line 1 - a is equal to b")
else:
    print ("Line 1 - a is not equal to b")
if ( a != b ):
    print ("Line 2 - a is not equal to b")
else:
    print ("Line 2 - a is equal to b")
if ( a < b ):
    print ("Line 3 - a is less than b")
else:
    print ("Line 3 - a is not less than b")
if ( a > b ):
    print ("Line 4 - a is greater than b")
else:
    print ("Line 4 - a is not greater than b")
a = 5;
b = 20;
if ( a <= b ):
    print ("Line 5 - a is either less than or equal to b")
else:
```

```
    print ("Line 5 - a is neither less than nor equal to b")
if ( b >= a ):
    print ("Line 6 - b is either greater than or equal to b")
else:
    print ("Line 6 - b is neither greater than nor equal to b")
```

Program 4: (Logical operators)

```
a=40
b='i'
if(a>50 and a<100):
    print("The value lies between 50 to 100")
else:
    print("The given condition is not satisfied")

if(b=='a' or b=='e' or b=='i' or b=='o' or b=='u'):
    print("The given character is a vowel")
else:
    print("The given character is a consonant")
```

```
x="true"
y=not x
print("Not of x is ",y)
```

Program 5: (Identity operators)

```
a = 20
b = 20
if ( a is b ):
    print ("Line 1 - a and b have same identity")
else:
    print ("Line 1 - a and b do not have same identity")
if ( id(a) != id(b) ):
    print ("Line 2 - a and b do not have same identity")
else:
    print ("Line 2 - a and b have same identity")
b = 30
if ( a is b ):
    print ("Line 3 - a and b have same identity")
else:
    print ("Line 3 - a and b do not have same identity")
```

Program 6: (Membership operators)

```
a = 10
b = 2
```

```
list = [1, 2, 3, 4, 5 ];
if ( a in list ):
    print ("Line 1 - a is available in the given list")
else:
    print ("Line 1 - a is not available in the given list")
if ( b not in list ):
    print ("Line 2 - b is not available in the given list")
else:
    print ("Line 2 - b is available in the given list")
a = 2
if ( a in list ):
    print ("Line 3 - a is available in the given list")
else:
    print ("Line 3 - a is not available in the given list")
```

Program 7: (Bitwise operators)

```
a = 60          # 60 = 0011 1100
b = 13          # 13 = 0000 1101
c = 0
c = a & b;      # 12 = 0000 1100
print ("Line 1 - Value of c is ", c)
c = a | b;      # 61 = 0011 1101
print ("Line 2 - Value of c is ", c)
c = a ^ b;      # 49 = 0011 0001
print ("Line 3 - Value of c is ", c)
c = ~a;         # -61 = 1100 0011
print ("Line 4 - Value of c is ", c)
c = a << 2;     # 240 = 1111 0000
print ("Line 5 - Value of c is ", c)
c = a >> 2;     # 15 = 0000 1111
print ("Line 6 - Value of c is ", c)
```


PYTHON INPUT AND OUTPUT

Python input:

Python input accepts the input only in the form of string..

Example1:

```
num=input("Enter the value for number")
print(num)
num1=input("Enter the value for number")
print(num1)
result=num+num1    #100100
print("Result is ",result)
```

Example2:

```
num=int(input("Enter the value for number"))
print(num)
num1=int(input("Enter the value for number"))
print(num1)
result=num+num1
print("Result is ",result)
```

Python output:

Example3:

```
print(1,2,3,4,5)
print(1,2,3,4,5, sep='*')
print(1,2,3,4,5, sep='#', end='@')
print(1,2,3, end='$')
print(5,6,7)
print(1,2,3, end="\n")
print(5,6,7)
```

Example4:

```
x=10
y=20
print("The value of x is {} and y is {}".format(x,y))
print("The value of x is {} and y is {}".format(y,x))
print("The value of x is {0} and y is {1}".format(100,200))
print("Integer value is {} Character value is {} String value is {}".format(1000,'R',"Anu"))
print("Integer value is {2} Character value is {0} String value is {1}".format(10,100,1000))
```

PYTHON CONDITIONAL EXECUTION

A statement that controls the flow of execution depending on some other condition. Python Conditional Execution statements can be classified as two major types namely,

- Ø Looping Statements
- Ø Decision Making Statements

LOOPING STATEMENTS:

Looping statements are the statements that execute one or more statements repeatedly for multiple number of times until the condition goes false. The looping statements available in python are

- ü for
- ü for loop with else
- ü while
- ü while loop with else
- ü break
- ü continue
- ü pass

FOR LOOP:

Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.

Example program 5a :

```
for letter in 'Python':    # First Example
    print ('Current Letter :', letter)
```

```
fruits = ['banana', 'apple', 'mango']
for a in fruits:          # Second Example
    print ('Current fruit :', a)
```

```
plants = ['Neem', 'Oak', 'Pine']
for index in range(len(plants)):
    print ('Current plant :', plants[index])
```

```
print ("Good bye!")
```

Example program 5b :

```
for x in range (0,3) :  
    print ('Loop execution %d' % (x))
```

FOR LOOP WITH ELSE :

Python supports to have an else statement associated with a loop statement

- If the else statement is used with a for loop, the else statement is executed when the loop has exhausted iterating the list.
- If the else statement is used with a while loop, the else statement is executed when the condition becomes false.

Example program 6:

```
lower = 1  
upper = 10  
# uncomment the following lines to take input from the user  
#lower = int(input("Enter lower range: "))  
#upper = int(input("Enter upper range: "))  
print("Prime numbers between",lower,"and",upper,"are:")  
for num in range(lower,upper + 1):  
    for i in range(2,num):  
        if (num % i) == 0:  
            break  
    else:  
        print ('%d is a prime number \n' %(num))
```

THE RANGE() FUNCTION:

To loop through a set of code a specified number of times, we can use the range() function. The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example program 7a :

```
for x in range(6):  
    print(x)
```

Example program 7b :

```
#using the start parameter  
for x in range(2, 6):  
    print(x)
```

Example program 7c :

```
#Increment the sequence with 3 (default is 1):  
for x in range(2, 30, 3):  
    print(x)
```

WHILE LOOP:

Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.

Example program 8 :

```
count = 0  
while (count < 9):  
    print( 'The count is:', count)  
    count = count + 1
```

WHILE LOOP WITH ELSE :

Example program 9:

```
count = 0  
while count < 5:  
    print (count, " is less than 5")  
    count = count + 1  
else:  
    print (count, " is not less than 5")
```

INFINITE LOOP:

A loop becomes infinite loop if a condition never becomes FALSE. You must use caution when using while loops because of the possibility that this condition never resolves to a FALSE value. This results in a loop that never ends. Such a loop is called an infinite loop.

An infinite loop might be useful in client/server programming where the server needs to run continuously so that client programs can communicate with it as and when required.

Example program 10 :

```
var = 1  
while var == 1 : # This constructs an infinite loop  
    num = input("Enter a number :")  
    print ("You entered: ", num)
```

NESTED LOOPS:

You can use one or more loop inside any another while, for or do..while loop.

Example program 11a :

```
i = 2
```

```

while(i < 100):
    j = 2
    while(j <= (i/j)):
        if not(i%j): break
        j = j + 1
    if (j > i/j) : print (i, " is prime")
    i = i + 1

```

Example program 11b :

```

for g in range(1, 6):
    for k in range(1, 3):
        print ("%d * %d = %d" % ( g, k, g*k))

```

BREAK STATEMENT:

Terminates the loop statement and transfers execution to the statement immediately following the loop.

Example program 12 :

```

for letter in 'Python':    # First Example
    if letter == 'h':
        break
    print('Current Letter :', letter)

```

```

var = 10                # Second Example
while var > 0:
    print ('Current variable value :', var)
    var = var -1
    if var == 5:
        break

```

CONTINUE STATEMENT:

Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

Example program 13a :

```

for letter in 'Python':    # First Example
    if letter == 'h':
        continue
    print ('Current Letter :', letter)

```

```

var = 10                # Second Example

```

```
while var > 0:
    var = var -1
    if var == 5:
        continue
    print ('Current variable value :', var)
```

Example program 13b :

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

PASS STATEMENT:

The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

Example program 14a :

```
for letter in 'Python':
    if letter == 'h':
        pass
    print( 'This is pass block')
    print ('Current Letter :', letter)
```

Example program 14b :

```
for letter in 'TutorialsCloud':
    if letter == 'C':
        pass
    print ('Pass block')
    print ('Current letter is:', letter)
```

PYTHON CONDITIONAL EXECUTION (continued)

DECISION STATEMENTS:

Decision making statements requires the programmer to specify one or more conditions to be evaluated or tested by the programmer. The decision making statements are

- ü simple if
- ü if.....else
- ü if....elif....else
- ü nested if

SIMPLE IF:

An if statement consists of a boolean expression followed by one or more statements.

Example program1 :

```
Rice_amount=1000
amount_per_qty=100
qty_purchased=int(input("Enter the quantity of rice purchased"))
if((qty_purchased*amount_per_qty)==Rice_amount):
    print("The calculation which you have made is correct for the quantity of 10 kgs
rice")
```

IF THEN ELSE:

An if statement can be followed by an optional else statement, which executes when the Boolean expression is FALSE.

Example program2 :

```
password=input("Enter the password: ")
if(password=='hello'):
    print("Your password has been accepted")
else:
    print("Sorry!!! You have entered a wrong password")
```

ELIF :

An if statement can be followed by any number of elif statement and a single else statement, which executes when the boolean expression is FALSE.

Example program 3 :

```
number=22
guess=int(input("Enter a number based on your guess: "))
if(guess==number):
    print("Congrats!!!!You guessed it!!!!!!")
elif(guess>number) and (guess != 0) :
    print("No, its little lower than your guess")
elif(guess<number) and (guess != 0) :
    print("No, its little greater than you guess")
else:
    print("The number you entered is zero")
```

NESTED IF:

We can use one if or else if statement inside another if or else if statement(s).

Example program 4 :

```
a=int(input("Enter a integer value"))
if(a>=0):
    if(a==0):
        print("The number you have entered is ZERO")
    else:
        print("POSITIVE number")
else:
    print("NEGATIVE number")
```


PYTHON FUNCTIONS

Python functions are techniques used to combine a set of statements within a program. Functions also let programmers compute a result-value and give parameters that serve as function inputs that may change each time the code runs. Functions are an alternative method of cutting-and-pasting codes, rather than typing redundant copies of the same instruction or operation; which further reduces the future work for programmers. They are the most basic structure of a program, and so Python provides this technique for code re-use.

Types of functions:

- Ø Built in functions.
- Ø User defined functions.

Syntax:

```
def functionname( parameters ):
    "function_docstring"
    Statements ()
```

Advantages Of Python Functions:

- Maximizing code reusability
- Minimizing redundancy
- Procedural decomposition
- Make programs simpler to read and understand

Defining a function:

You can define functions to provide the required functionality. Here are simple rules to define a function in Python.

- ü Function blocks begin with the keyword ‘**def**’ followed by the function name and parentheses (()).
- ü Any input parameters or arguments should be placed within these parentheses.
- ü You can also define parameters inside these parentheses.
- ü The code block within every function starts with a colon (:) and is indented.
- ü The statement return [expression] exits a function, optionally passing back an expression to the caller.

- ü A return statement with no arguments is the same as return None.

Calling a Function:

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt.

Functions can be called in different ways. They are

- ü Providing the mandatory argument only
- ü Providing one of the optional argument
- ü By giving all the arguments

Example1:

Function definition is here

```
def printme( str ):
    print (str);
```

Now you can call printme function

```
printme("I'm first call to user defined function!!!!!!");
printme("Again second call to the same function!!!!!!");
```

Example 2:

```
def my_function(fname):
    print(fname + " Reference")
```

```
my_function("Email")
my_function("Gmail")
my_function("Hotmail")
```

Return Statement:

The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return none.

Example 3:

Function definition is here

```
def sum( arg1, arg2 ):
# Add both the parameters and return them."
    total = arg1 + arg2
    print ("Inside the function : ", total)
    return total;
# Now you can call sum function
total = sum( 10, 20 );
print ("Outside the function : ", total)
```

Example 4:

```
def my_prog(x):
    a=5 * x
    return a
```

```
print(my_prog(3))
print(my_prog(5))
print(my_prog(9))
```

PYTHON FUNCTIONS (continued)

Scope of Variables:

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable. The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python:

- Ø Global variables
- Ø Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope. This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope.

Example 5:

```
total = 0; # This is global variable.
# Function definition is here
def sum( arg1, arg2 ):
    # Add both the parameters and return them."
    total = arg1 + arg2; # Here total is local variable.
    print ("Inside the function local total : ", total)
    return total;

print("Local call of my function is ",sum(100,200))
print("Outside the Function :",total)
```

Example 6:

```
def fact(n):
    x=1;#This is a local variable
    for i in range(1,n+1):
        x=x*i
    return x;

m=int(input("Enter a number"))
print("Factorial value is {}".format(fact(m)))
```

Example 7:

```
def reverse(n):
    s=0
```

```
while (n!=0):
    s=s*10+(n%10)
    n=n//10
return s;
```

```
m=int(input("Enter a number "))
print("Reverse is {}".format(reverse(int(m))))
```

Python Function Parameters:

Information can be passed to functions as parameter. Parameters are specified after the function name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.

- Default arguments
- Required arguments
- Keyword arguments
- Variable-length arguments

Default Arguments:

A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument. If we call the function without parameter, it uses the default value.

Example 7:

```
def my_function(country = "Norway"):
    print("I am from " + country)
```

```
my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

Example 8:

```
# Function definition is here
def printinfo( name, age = 35 ):
    print ("Name: ", name)
    print ("Age ", age)
```

```
# Now you can call printinfo function
printinfo( age=50, name="miki" );
```

```
printinfo( name="Priya" );
```

Required Arguments:

Required arguments are the arguments passed to a function in correct positional order. Here, the number of arguments in the function call should match exactly with the function definition.

Example 9:

```
# Function definition is here
```

```
def printme( str ):  
    print (str);
```

```
# Now you can call printme function
```

```
printme("Hiii....");  
printme("How are you????");
```

Example 10:

```
# Function definition is here
```

```
def fun_call(name,salary):  
    print (name,salary);
```

```
# Now you can call fun_call function
```

```
fun_call("Anu",12000);  
fun_call(20000,"Ram");
```

Keyword Arguments:

Keyword arguments are related to the function calls. When you use keyword arguments in a function call, the caller identifies the arguments by the parameter name. This allows you to skip arguments or place them out of order because the Python interpreter is able to use the keywords provided to match the values with parameters.

Example 11:

```
# Function definition is here
```

```
def printme( str ):  
    print (str);
```

```
# Now you can call printme function
```

```
printme( str = "We live in chennai");
```

Example 12:

```
# Function definition is here
def printinfo( name, age ):
    print( "Name: ",name)
    print( "Age: ", age)
    return;
# Now you can call printinfo function
printinfo( age=100, name="abcde" );
```

Variable Length Arguments:

Sometimes we may need to process a function for more arguments than that of the specified size while defining the function. These arguments are called variable-length arguments and are not named in the function definition, unlike required and default arguments.

Example 13:

```
# Function definition is here
def printinfo( *vartuple ):
    for var in vartuple:
        print( var)

# Now you can call printinfo function
printinfo( 10 );
printinfo( 70, 60, 50,"Chennai","Banglore");
```

Python Recursion Function:

Recursion is defined as a process of calling the function again and again on its own.

Example 13:

```
def fibo(n):
    a =0
    b =1
    for i in range(0, n):
        temp = a
        a = b
        b = temp + b
    return a
for c in range(0,5):
```

```
print(fibo(c))#Function call
```

Lambda Functions:

In python, the keyword lambda is used to create what is known as anonymous functions. These are essentially functions with no pre-defined name. They are good for constructing adaptable functions, and thus good for event handling.

Example 14:

```
keyboard = lambda i: i*2  
print(keyboard(2))  
myfunc = lambda x,y: x*y  
print(myfunc(3,6))
```

Example 15:

```
def myfunc(n):  
    return lambda i: i*n  
  
doubler = myfunc(2)  
tripler = myfunc(3)  
val = 11  
print("Doubled: " + str(doubler(val)) + ". Tripled: " + str(tripler(val)))
```

Example 16:

```
# Function definition is here  
sum = lambda arg1, arg2: arg1 + arg2;  
# Now you can call sum as a function  
print ("Value of total : ", sum( 10, 20 ))  
print ("Value of total : ", sum( 20, 20 ))
```


PYTHON MODULES

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and refer. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Roles of python module:

- Ø **Code Reuse:** Modules let programmers save source code in files permanently & hence the codes in module files are persistent. Module codes can rerun as many times as required.
- Ø **System Namespace Partitioning:** In Python, the highest-level organizational unit of code is the Modules. Although they are just packages with names - these packages are self-contained. Like that of a local scope of functions, modules help to avoid name clashes across the program. So in a module, all the objects and codes are implicitly enclosed or bounded. Hence, modules are the natural programming tools used to group system components.
- Ø **Shared Service or data Implementation:** From the operational point of view, modules are also used to implement components that are shared across systems.

For example, if a programmer needs to provide a global object which will be used by more than one files or function blocks, then the code can be used as a module that can be imported by several files.

The *import* Statement:

syntax:

import module1,module2,... module N

Example for import statement:

```
def print_func( par ):  
    print ("Hello : ", par)  
    return
```

```
import mod1          #here mod1 represents the saved name of function definition
```

```
mod1.print_func("Zara")
```

```
import mod1 as m1      #import by renaming the module  
m1.print_func("Licy")
```

The from...import Statement:(otherwise called python package)

syntax:

```
from module_name import name1[, name2[, ... name N]]
```

Example for from ... import statement:

```
def add(a,b):  
    c=a+b;  
    return(c)
```

```
def sub(d,e):  
    c=d+e;  
    return (c)
```

```
def multi(f,g):  
    c=f*g;  
    return (c)
```

```
def divide(h,i):  
    c=h/i;  
    return (c)
```

```
from mod2 import multi  
result=multi(int(100),int(200))  
print("Result is ",result)
```

The from...import * Statement:

It is also possible to import all names from a module into the current namespace by using the import * statement. This provides an easy way to import all the items from a module into the current namespace however, this statement should be used sparingly.

syntax:

```
from modname import *
```

Example for from ... import * statement:

```
def Pots():  
    print ("I'm Pots Phone")  
def Isdn():  
    print("I'm Isdn phone")  
def G3():  
    print("I'm G3 Phone")
```

```
from res import *;  
Pots()  
Isdn()  
G3()
```

The dir() Function

The dir() built-in function returns a sorted list of strings containing the names defined by a module. The list contains the names of all the modules, variables and functions that are defined in a module.

Example for dir():

```
# Import built-in module math  
import math  
content = dir(math)  
print (content);
```

The reload() Function

When the module is imported into a script, the code in the top-level portion of a module is executed only once. Therefore, if you want to re-execute the top-level code in a module, you can use the reload() function. The reload() function imports a previously imported module again.

syntax:

```
reload(module_name)
```

DATA STRUCTURES

Python data structures involves sorting and searching.

Sorting:

Sorting is defined as the process of arranging the elements either in ascending or in descending order.

Some of the sorting techniques are:

- Bubble sort
- Selection sort
- Insertion sort

Bubble Sort:

Bubble sort is used to compare N elements present in the memory. It compares all the element one by one and sort them based on their values. On each iteration the largest element in the list bubbles up towards the last place.

Example of Bubble sort:

```
#bubble sort
#N=[54,26,93,17,77,31,44,55,20]
#N=[83,25,50,70,14,74,67,90]
N=[22,43,21,13]
for i in range(0,len(N)):
    for j in range (0,len(N) -1):
        if N[j] > N[j+1]:
            temp=N[j]
            N[j]=N[j+1]
            N[j+1]=temp
print ("Sorted values are ",N)
```

Selection sort:

Selection sort is also used in sorting the elements either in ascending or descending order. In selection sort the first element in the list is selected and it is compared repeatedly compared with all other elements in the list. If any element is smaller than the selected element, then both are swapped. Then we select the second element in the list, it is compared with all other elements in the list and swapped if it is smaller. The same procedure is repeated.

Example of Selection sort:

```
#selection sort
N=[83,25,50,70,14]
for i in range(len(N)-1):
    minpos=i
    for j in range (i+1,len(N)):
        if N[j] < N[minpos]:
            minpos=j
    temp=N[minpos]
    N[minpos]=N[i]
    N[i]=temp

print ("Sorted values are ",N)
```

Insertion sort:

In insertion sort algorithm, each iteration moves an element from unsorted position to the sorted position until all the elements are sorted in the list.

Example of Insertion sort:

```
#Insertion sort
N=[83,25,50,70,14,74,67,90]
for i in range( 1,len(N)):
    curval=N[i]
    curpos=i
    while curpos >0 and N[curpos - 1] > curval:
        N[curpos] = N[curpos - 1]
        curpos = curpos - 1
    N[curpos] = curval

print ("Sorted values are ",N)
```

Searching:

Searching is a process of finding a particular value from the list of given items. There are two types of searching algorithms they are:

- Linear or Sequential Search
- Binary Search

Linear Searching:

Example of Linear search:

```
N=[13,5,7,4,12,8,9,11,23,45,56]
```

```
flag=True
```

```
while flag:
```

```
    find_num=int(input("Enter the number to be found: "))
```

```
    for i in range(0, len(N)):
```

```
        if N[i]== find_num :
```

```
            print("Number Found")
```

```
            print("Position is ",i+1)
```

```
            flag=False
```

```
            break;
```

```
    else:
```

```
        print("The entered number is not present in the list")
```

```
        flag=False
```

```
        break
```

Binary Searching:

Binary search can be done only in a sorted list of elements. It can be done by repeatedly dividing the search interval into half. Begin with the interval covering the whole list. If the value of the search key is less than the item in the middle of the interval, narrow the interval to lower half. Otherwise narrow it to the upper half. Perform the same task until the elements of the list are sorted.

Example of Binary search:

```
N=[23,87,99,123,345]
```

```
while True:
```

```
    find_num=int(input("Enter the number to be found: "))
```

```
    first=0
```

```
    last=len(N)-1
```

```
    while first <= last:
```

```
        midpoint=(first+last)//2
```

```
        if N[midpoint] == find_num:
```

```
            print("Found")
```

```
            break
```

```
        else:
```

```
            if find_num < N[midpoint]:
```

```
        last=midpoint - 1
    else:
        first= midpoint + 1
else:
    print("The given value is not present in the list")
```

PYTHON SEQUENCES

- ü The most basic data structure in Python is the **sequence**. Each element of a sequence is assigned a number - its position or index. The first index is zero, the second index is one, and so forth.
- ü Python has six built-in types of sequences, but the most common ones are lists and tuples.
- ü Some of the operations of sequence include indexing, slicing, adding, multiplying, and checking for membership. In addition, Python has built-in functions for finding the length of a sequence and for finding its largest and smallest elements.

Python Lists

Python list is a mutable object. The list is a most versatile datatype available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list:

The list can be created by placing all the items inside a square bracket [], separated by commas. It can have any kinds of datatypes.

Accessing Values in Lists:

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index.

Updating Lists:

We can update single or multiple elements of lists by giving the slice on the left-hand side of the assignment operator, and can add to elements in a list with the `append()` method.

Deleting List Elements

To remove a list element, we can use either the `del` statement if we know exactly which element(s) we are to delete or the `remove()` method if you do not know.

Example program involving List Basic operations:

```
#List creation
#normal list
list1 = ['apple', 'orange', 'mango', 'peach'];
list2 = [1, 2, 3, 4, 5, 6, 7 ];
list3=['a','e','i','o','u']
print("list3[2] is ",list3[2])
print("list3[4] is ",list3[4])
```



```

#nested list
list4=["Hi", "Students", "Welcome", "To", "PYTHON", [list3]]
print("The values of list4 are ",list4)
print("Value of list4[index 2]",list4[2][0])

#Displaying the values of the list
print ("list1[0]: ", list1[0])
print ("list2[1:5]: ", list2[1:5])

#updating the values of the list
print ("Value available at index 2 in list2 : ",list2[2])
list2[2] = 200;
print ("New value available at index 2 in list2 is : ",list2[2])

#deleting the values of the list
del list1[2];
print ("After deleting value at index 2 : ",list1)

#positive and negative indexing
print("[3:5] index value of list4 is ",list4[3:5])
print("[4:] index value of list4 is ",list4[4:])
print("[:] index value of list4 is ",list4 [:])
print("[ :-3] index value of list4 is ",list4[ :-3])
print("[-1] index value of list4 is ",list4[-1])
print("[-5] index value of list4 is ",list4[-5])
list5=list1+list2+list3    #concatenation
print("Concatenated value is ",list5)
print("Repetition value is ",list3 *2)    #Repetition operation

```

Built-in List Functions

Python includes the following list functions

len(list)	Gives the total length of the list.
max(list)	Returns item from the list with max value.
min(list)	Returns item from the list with min value.
list(seq)	Converts a iterable object into list.
all()	Returns TRUE if all the elements of the list are true.
any()	Return TRUE if any element of the list is true. If the list is empty, return false.
Enumerate()	Returns an enumerate object. It contains the index and values of the items of the list.
Sorted()	Returns a sorted list.
sum()	Returns the sum of all the elements.

Python List Methods:

append()	adds an element to the end of the list.
extend()	add all the element of the list to another list.
insert()	inserts an item from the list.
remove()	Removes an item from the list.
pop()	Removes and returns an element at the given index.
clear()	Removes all the item from the list.
index()	Returns the index of the first matched item.
count()	Returns the count of number of items.
sort()	Sorts items in ascending order.
reverse()	Reverses the items in the list.
copy()	Returns a shallow copy of the list.

Example program invloving List functions:

#List creation

```
list1, list2 = [1000, 'Python', 'Application'], [2018, 'List Chapter']
```

```
list3, list4 = ['xyz', 'Monty', 'abc'], [456, 700, 200]
```

#converting tuple to list

```
Tuple_format = (123, 'xyz', 'zara', 'abc');
```

```
List_format = list(Tuple_format)
```

```
print("List format is " , List_format)
```

#List length

```
print ("First list length : ", len(list1))
```

```
print ("Second list length : ", len(list2))
```

#Maximum value of the list

```
print ("Max value element : ", max(list3))
```

#Minimum value of the list

```
print ("Min value element : ", min(list3))
```

#append function

```
list4.append( 2009 );
```

```
print ("Updated List : ", list4)
```

#displaying the values of the list

```
print("List1 is ",list1)
```

```
print("List2 is ",list2)
```

```
print("List3 is ",list3)
```

```
print("List4 is ",list4)
```

```
#Creating a new list
```

```
new_list=list1+list2+list3+list4
```

```
print("New_List is ",new_list)
```

```
#Count of list items
```

```
print ("Count of Application : ", list1.count('Application'))
```

```
#extend function
```

```
list5 = [2009, 'Zathura'];
```

```
new_list.extend(list5)
```

```
print ("Extended List : ", new_list )
```

```
#index function
```

```
print ("Index for xyz : ", list3.index( 'xyz' ))
```

```
list1.insert( 3, 2009)
```

```
print ("List after insertion : ", list1)
```

```
#pop function
```

```
print ("Popped item of the list is : ", new_list.pop())
```

```
print ("Popped item of the new_list(2) is : ", new_list.pop(2))
```

```
#remove function
```

```
list1.remove(1000);
```

```
print ("List1 after removing the items : ", list1)
```

```
list3.remove('abc');
```

```
print ("List3 after removing the items : ", list3)
```

```
#list reverse
```

```
new_list.reverse();
```

```
print ("Reverse of the List is : ", new_list)
```

```
#sort the list items
```

```
list4.sort();
```

```
print ("Sorted items of the List4 is : ", list4)
```

Python Tuple

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple:

The tuple can be created by placing all the items inside a parentheses (), separated by commas. It can have any kinds of datatypes.

Accessing Values in Tuple:

The values of the tuple can be accessed in the same way that the tuple does. Some of the ways to access the elements of the tuple are indexing, slicing, negative slicing, etc.,

Updating Tuples:

Tuples are immutable which means the tuple cannot be updated or changed. But it is possible to take portions of existing tuples to create new tuples.

Deleting Tuple Elements:

Removing individual tuple elements is not possible. To explicitly remove an entire tuple, just use the del statement.

Example program involving Tuple Basic operations:

```
#tuple creation
```

```
atuple = ('Samsung', 'Oppo', 2017, 2018);
```

```
btuple = (100, 200, 300, 400, 500, 600, 700 );
```

```
ctuple = ('P','Y','T','H','O','N')
```

```
#nested tuple
```

```
dtuple = ((1,2,3),(4,5,6),(7,8,9))
```

```
#indexing
```

```
print ("atuple[0]: ", atuple[0])
```

```
print("atuple[0][4] :",atuple[0][4])
```

```
#slicing
```

```
print ("btuple[1:5]: ", btuple[1:5])
```

```
print ("btuple[2:]: ",btuple[2:])
```

```
#negative indexing
```

```
print ("btuple[2:-2]: ",btuple[2:-2])
```

```
print ("ctuple[-3]: ", ctuple[-3])
```

```
# Following action is not valid to update a tuple
```

```
# atuple[0] = 100;
# So let's create a new tuple as follows
etuple = atuple + btuple;
print (etuple);

#concatenation
print("Concatenated value is : ", (etuple+dtuple))

#Repetition operation
print("Repetition operation is : ", ("PYTHON" *4))

#cannot delete a specific value
del atuple[3]
print(atuple)
```

Built-in Tuple Functions

Python includes the following list functions

len()	Gives the total length of the tuple.
max()	Returns item from the tuple with max value.
min()	Returns item from the tuple with min value.
tuple()	Converts a iterable objects into tuple.
all()	Returns TRUE if all the elements of the tuple are true.
any()	Return TRUE if any element of the tuple is true. If the tuple is empty, return false.
Enumerate()	Returns an enumerate object. It contains the index and values of the items of the tuple.
Sorted()	Returns a sorted tuple.
sum()	Returns the sum of all the elements.

Python Tuple Methods:

Methods to add or remove the element from the tuple is not possible.

count(x)	Returns the no of items present in the tuple
index()	Returns the index of the first item that is equal to x

Example program invloving Tuple Functions:

```
tuple1, tuple2 = ('vijay', 'Ram', 'vijay', 'Shankar', 'vijay'), (456, 786,9867,1098)
```

```
#length function
print ("First tuple length : ", len(tuple1));
print ("Second tuple length : ", len(tuple2));
```

```
#maximum element in the tuple
print ("Max value element : ", max(tuple1));
print ("Max value element : ", max(tuple2));
```

```
#minimum element in the tuple
print ("min value element : ", min(tuple1));
print ("min value element : ", min(tuple2));
```

```
#converting list to tuple
aList = [123, 'xyz', 'zara', 'abc'];
aTuple = tuple(aList)
print ("Tuple elements : ", aTuple)
```

```
#Index function
print("Index value of Ram is ",(tuple1.index('Ram')))
```

```
#count function
print("Count of the name vijay present in tuple1 is ",(tuple1.count('vijay')))
```

```
#membership operation
print(9867 in tuple2)
```

```
#iterating through the tuple
for person in tuple1:
    print("Hello ", person)
```

Python Arrays

Python arrays is used to store the values of same datatype. Arrays is similar to that of python list but it completely does not satisfy the concept of list. Under python arrays only the similar datatype can be accepted. To use python arrays we need to import the “array” module, this is because array is not a fundamental datatype like string, integer, etc.,

Syntax:

```
from array import *
```

Array Declaration Syntax:

```
Arrayidentifiername=array('typecode',[initializers])
```

Example:

```
from array import *  
array_name = array('i',[100,200,300,400,500])
```

Typecodes and its allocation:

Type code	C type	Python Type	Minimum size of bytes
'c'	Char	character	1
'b'	signed char	int	1
'B'	unsigned char	int	1
'u'	PY_UNICODE	unicode character	2
'h'	signed short	int	2
'H'	unsigned short	int	2
'i'	signed int	int	2
'I'	unsigned int	long	2
'l'	signed long	int	4
'L'	unsigned long	long	4
'f'	Float	float	4
'd'	Double	float	8

Methods existing in arrays:

Methods	Purposes
append()	Appends any value into the array.
insert()	Inserts a value into the array.
extend()	Extends the content of the array.
fromlist()	Adds an item from the list to an array.
remove()	Removes an element from an array.
pop()	Removes the last element of the array.
index()	Fetchs an element from the particular index position.

reverse()	Reverses the array.
buffer_info()	Provides the buffer information(provides the start address and number of elements in the array).
count()	Counts the number of elements in the array.
tostring()	Converts array to string.
tolist()	Converts array to list.
fromstring()	Appends a string to character array

Example program involving python arrays:

```
#Python Arrays
#declaration of an array
from array import *
array_info = array('i', [100,200,300,400,500])
for i in array_info :
    print("Array_info is " ,i)

#accessing individual element of an array
print("Array_info[2] is ",array_info[2])
print("Array_info[4] is ",array_info[4])

#appending the values
array_info.append(600)
print("Array_info after appending a value is " ,array_info)

#insertion of the values
array_info.insert(6,700)
array_info.insert(6,650)
print("Array_info after inserting a value is " ,array_info)

#extending the array items
a= array('i', [800,900,1000])
array_info.extend(a)
print("Array_info after extending the values are " ,array_info)

#fromlist()
list=[1100,1200,1300,1400]
array_info.fromlist(list)
print("Array_info after using the fromlist method is " ,array_info)

#remove an element from the array
array_info.remove(650)
```



```
array_info.remove(list[0])  
print("Array_info after using the fromlist method is ",array_info)
```

```
#to pop an element  
print("Array_info after popping an element is ",array_info.pop())  
print("Array_info after popping an element in 3rd position is "  
,array_info.pop(3))
```

```
#index of the element  
print("Index of the element 1200 is ", array_info.index(1200))
```

```
#reverse of the element  
array_info.reverse()  
print("Reverse of array_info is ",array_info)
```

```
#buffer_info  
print("Buffer information of array_info is ",array_info.buffer_info())
```

```
#count method  
print("Count of the element 500 is ",array_info.count(500))
```

```
#convert array to list  
result=array_info.tolist()  
print("character_array to list is ",result)
```

Python Strings

Python string is a sequence of character. A character is simply a symbol. For example, the English language has (26) characters. Computer do not deal with characters, they deal with numbers (Binary). Even though you may see characters on your screen, internally it is stored and manipulated as the combination of 0's and 1's.

This conversion of character to a number is called encoding, and the reverse process is decoding. ASCII and Unicode are some of the popular encoding used.

In python string is a sequence of Unicode character. Unicode was introduced to include every character in all languages and bring uniformly in encoding. You can learn more about Unicode from [here](#).

How to create String ?

```
S='Hai guys'
print(S)
St="Welcome"
print(St)
Str1="""CSC Welcomes U"""
print(Str1)
```

How to access character in a string?

We can access individual characters using indexing and a range of characters using slicing. Index starts from 0. Trying to access a character out of index range will raise an indexing error. The index must be an integer.

```
St='CSC Computer'
Print('Given string is',St)
```

```
Print('st[0]',st[0])
```

```
Print('st[-1]', st[-1])
Print('st[1:5]', st[1:5])
Print('st[5:-2]', st[5:-2])
```

Python string are immutable.

```
Str1='Python'
Str1[3]='a'
```

```
Del str1[3]
```

Python String Operation

There are many operations that can be performed with string which makes it one of the most used datatypes in python.

The + operator is used to join the two string in Python.

```
Word1="Hello"
Word2="World"
Print("The combined string is",Word1+Word2)
Print(Word1*3)
```

Iteration through String

Using for loop we can iterate through a string. Here is an example to count the number of 'o' in a string.

```
count =0
for q in 'Hello World':
    if(q=='o'):
        count+=1
print('o present',count,'times')
```

Format() Method for formatting String

The format() method that is available with string object is very versatile and powerful in formatting strings. Format strings contains curly braces{} as placeholders or replacement fields which gets replaced.

We can positional arguments or keyword arguments to specify the order.

Default Order

```
Order="{},{ }and{}".format('CSC','Computer','Education')
Print(order)
```

Positional Order

```
Order="{2},{0}and{1}".format('CSC','Computer','Education')
Print(order)
```

Common Python String Method

Capitalize()

```
St='csc computer'
Print(St.Capitalize())
```

Center()

```
St='csc computer'
St.center(4,"#")
```

Count()

```
St='I am an Indian'
St.count("an",0,len(St))
```

Find()

Syntax:

```
String.find(substring,begindex=0,endindex=len(string))
```

St='I am an Indian'

Common Python String Methods

Str.capitalize()

str="csc computer education"

str.capitalize()

string.center(width[,fillchar])

str="csc computer education"

str.center(40,"#")

string.count(substring,startindex=0,endindex=len(string))

str1="I am an Indian"

str1.count("an",0,len(str))

string.find(substring,begindex=0,endindex=len(string))

str1="I am an Indian"

str1.count("an",0,len(str))

str1.find("an",0,len(str))

str1.find("In",0,len(str))

str.replace(old,new[,max])

str1="I am an Indian"

str1.replace("an","hi")

isaplha()

str1="I am an Indian"

str1.isaplha()

isdigit()

num=234

num.isdigit()

islower()

str1="indian"

```
str1.islower()
```

```
isspace()
```

```
str="  "
```

```
str.isspace()
```

```
istitle()
```

```
str1="I am an Indian"
```

```
str1.istitle()
```

```
str1="I AM AN INDIAN"
```

```
str1.istitle()
```

```
#Program1
```

```
constants="hgjkdlsVBNMTY"
```

```
vowels="AaEeliOoUu"
```

```
digits="1234567890"
```

```
ccount=0
```

```
cvowels=0
```

```
dcount=0
```

```
ocount=0
```

```
str1=input("Enter some text")
```

```
for i in str1:
```

```
    if i in constants:
```

```
        ccount=ccount+1
```

```
    elif i in vowels:
```

```
        cvowels=cvowels+1
```

```
    elif i in digits:
```

```
        dcount=dcount+1
```

```
    else:
```

```
    ocount=ocount+1  
print("No of Constants",ccount)  
print("No of vowels",cvowels)  
print("No of digits",dcount)  
print("others",ocount)
```

PYTHON SEQUENCES

Python Set

Python set is an unordered collection of items. Every element is unique and must be immutable object.

Creating a Set:

The set can be created by placing all the items inside a curly braces {}, separated by commas or by using built in function set().

Example program for Set:

```
#Set creation
```

```
# Set of Integers
```

```
set1={2,5,7}
```

```
print(set1)
```

```
#Different datatypes in set
```

```
set2={16,"book",(2,3,4),4.9,8+1j}
```

```
Print(set2)
```

```
#Set will not print the duplicates
```

```
set3={1,2,3,4,5,5,6,7}
```

```
print(set3)
```

```
#Empty set creation
```

Empty curly braces {} will make an empty dictionary in Python.

```
a={}
```

```
type(a) #<class 'dict'>
```

```
#Initialize with set()
```

```
a=set()
```

```
type(a) #<class 'set'>
```

```
#Add element in set
```

```
set1={2,5,7}
```

```
set1.add(6)
```

```
print(set1)
```

```
#Add multiple element in set
```

```
set1.update([4,3,7])
```

```
print(set1)
```



```
#Add list and set
set1.update([12,36],{8,7,5})
print(set1)

# Remove elements from a set
set3={1,2,3,4,5,6,7,8}
print(set3)

set3.discard(3)
print(set3)

set3.remove(5)
print(set3)

set3.clear()
print(set3)
```

Python Set Operation

Sets can be used to carry out mathematical set operations like union, intersection, difference and symmetric difference. We can do this with operators or methods .

Example 1:- Set Union:

```
sa={2,3,4,5}
sb={2,3,8,6}

#Using | operator
print(sa|sb)

#Using union() method
print(sa.union(sb))
```

Example 2:-Set Intersection:

```
sa={2,3,4,5,7}
sb={2,3,8,6,1}

#Using & operator
```

```
print(sa&sb)
```

```
#Using intersection() method  
print(sa.intersection(sb))
```

Example 3:-Set Difference

```
sa={2,3,4,5,7}  
sb={2,3,8,6,1}
```

```
#Elements unique to sa  
Print("Elements only in sa set")
```

```
#Using - operator  
print(sa-sb)
```

```
#Using difference() method  
print(sa.difference(sb))
```

Example 4:-Set Symmetric Difference

```
sa={2,3,4,5,7}  
sb={2,3,8,6,1}
```

```
#Using ^ operator  
print(sa^sb)
```

```
#Using symmetric_difference() method  
print(sa.symmetric_difference (sb))
```

Python Dictionary

Python dictionary is an unordered collection of items. While other compound data types have only value as an element, a dictionary has a key : value pair.

Creating a Dictionary:

Creating a dictionary is as simple as placing items inside curly braces {} separated by comma.

An item has a key and corresponding value expressed as a pair, key : value. While values can be of any data type and can repeat, key must be of immutable type (string, number or tuple with immutable elements) and must be unique.

```
#Dictionary creation
dict1={'Eno':24195,'Name':'Sujay'}
print(dict1['Eno'])
print(dict1['Name'])
```

Add elements in Dictionary

Dictionary are mutable. We can add new items or change the values of existing items using assignment operator.

```
#Update value
Dict1['Eno']=25000
Print(Dict1)
```

```
Dict1['Course']='ADPP'
Print(Dict1)
```

```
#Delete elements in Dictionary
dict2={1,2,3,4,5,6,7,8,9,10}
print(dict2.pop())
```

```
dict2.clear()
```

```
del dict2
```

Dictionary Comprehension

Dictionary comprehension is an elegant and concise way to create new dictionary from an iterable in python.

```
squares={x:x*x for x in range(6)}  
print(squares)
```

Dictionary program to print only the odd numbers

```
Odd={x:x*x for x in range(11) if x%2==1 }  
Print(Odd)
```

Dictionary Membership test

We can test if a key is in a dictionary or not using the keyword in. Notice that membership test is for key only, not for values

```
Num={1:1,2:4,3:9,5:25,7:49}
```

```
Print(2 in num)  
Print(4 not in num)  
Print(7 not in num)
```

PYTHON FILE I/O

File is used in storing the related data on a non-volatile memory (Hard-disk) for further references. When we want to read from or write to the file we need to initially open it, finally when all the processing of the data is done it need to be closed.

Types of files available:

- Ø Text file (Default mode)
- Ø Binary file (Non-text file like images or exe file)

Python File operations are:

- Ø Opening a file
- Ø Reading or writing a file
- Ø Closing a file

Syntax to be followed:

File opening:

Ø file object = open("file_name" ," access_mode")

file_name: The file_name argument is a string value that contains the name of the file that you want to access.

access_mode: The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc.

Python file modes:

Mode	Description
'r'	Read mode.
'w'	Write mode.
'x'	Exclusive creation. If the file is already open then the operation fails.
'a'	Append mode.
't'	Text mode.
'b'	Binary mode.
'+'	Opening file for updating(reading & writing)

File closing:

Ø file object.close()

Writing to a file:

Ø fileobject.write(string); #To write data into the file we can either use write 'w', appened 'a'mode.

Reading from a file:

Ø file object.read([count]); #helps in reading a string based on the count
Ø file object.readline() #helps in reading one line. By calling readline() two times, it is possible to read the first two lines.

File Positions:

Ø file object.tell() #tells the current position within the file.
Ø file object.seek(offset, from) """The offset argument indicates the number of bytes to be moved. The from argument specifies the reference position from where the bytes are to be moved. If from is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position."""

Sample programs:

Program 1:(To open, close and write a data to a file)

```
# Open a file
a = open("one.txt", "w")
a.write( "Python is a great language!!!!!!.....Yeah its great!!!!!!.....");

# Close file
a.close()
```

Program 2a:(To open, close and read a data from the file)

```
# Open a file
name = open("one.txt", "r+")
str = name.read(20);
print ("The String that is read from the file is : ", str)

# Close file
name.close()
```

Program 2b:(To open, close and read a data from the file)

```
# read the entire file as one string(type 1)
```

```
with open('one.txt') as f:
    data = f.read()
    print(data)
```

```
# Iterate over the lines of the File(type 2)
with open('one.txt') as f:
    for line in f :
        print(line, end=' ')
```

Program 3:(File Positions)

```
# Open a file
r = open("one.txt", "r+")
str = r.readline();
print ("The string which is read is : ", str)
```

```
# Check current position
position = r.tell();
print ("Current file position : ", position)
```

```
# Reposition pointer at the beginning, mid and end
position = r.seek(0, 0);
print ("After using seek(0,0) the string value is : ", position)
position = r.seek(0, 1);
print ("After using seek(0,1) the string value is : ", position)
position = r.seek(0, 2);
print ("After using seek(0,2) the string value is : ", position)
```

```
# Close file
r.close()
```

Program 4:(To open, close and read a data from the file using loop)

```
# Open a file
a = open("smile.txt", "w")
a.write( "Childhood means simplicity..Look at the world with the child's eye -
its beautiful!!!!!!");
print("The content has been updated...")
```

```
# Close file
a.close()
```

```
f = open("smile.txt", "r")
for x in f:
    print(x)
```

Program 5:

#create a text file and write text into it

```
fwrite=open("two.txt","w")
```

```
ch="y"
```

```
while(ch=="y" or ch=="Y"):
```

```
    text=input("Enter a string value :")
```

```
    fwrite.write(text)
```

```
    fwrite.write("\n")
```

```
    ch=input("Would you like to continue any more!!!! If yes, type Yes else type No")
```

```
fwrite.close()
```

#To read the contents of the file(Type 1)

```
with open('two.txt') as fread:
```

```
    data = fread.read()
```

```
print(data )
```

```
fread.close()
```

#To read the contents of the file(Type 2)

```
with open('two.txt') as fread:
```

```
    for i in fread:
```

```
        print(len(i))
```

```
        print(i )
```

```
fread.close()
```

Program 6:(Writing Number to a File)

#writing numbers into a file

```
import random
```

```
f=open("integers.txt","w")
```



```

for count in range(500):
    number=random.randint(1,500)
    f.write(str(number) +"\n")
f.close()

```

Python File Methods:

Method	Description
close()	Closes an opened file.
fileno()	Returns an integer number of the file.
flush()	Flushes the write buffer of the file.
isatty()	Returns true if the file buffer is interactive.
read(n)	Reads n characters from a file.
readable()	Returns true if the file is in read mode.
readline(n=-1)	Reads a single line from the file.
readlines(n=-1)	Reads a list of lines from a file.
seek(offset,from=SEEK_SET)	Change the file position to the offset bytes.
seekable()	Returns true if the file supports the random access.
tell()	Returns the current file location.
truncate(size=None)	Resize the file stream to the file buffer.
writable()	Returns true if the file is in read mode.
write(s)	Writes a string to a file.
writelines(lines)	Writes a list of lines to the file.

Binary Files in python:

The python file requires the pickle module to be imported. Pickle module consist of 2 main methods.

- ü dump() – dumps an object to an file object
- ü load() – loads an object from a file object

Program 7:(Binary files using dictionary)

```

#storing and retriving data as dictionary
import pickle
file1=open("students.txt","bw")
course1={9001:['Priya','HDCA',7600],9002:['Geetha','DMO',2500]}
pickle.dump(course1,file1)
file1.close()
file1=open("students.txt","br")
course1=dict(pickle.load(file1))

```

```

file1.close()
r=int(input("Enter Enrollment number: "))
for regno,cc in course1.items():
    if regno==r:
        flag=True
        while flag:
            print("Student Reg No is ",regno)
            print("Student Name is ",cc[0])
            print("Enter course ",cc[1])
            print("Balance Fee ",cc[2])
            amt=int(input("Enter the amount : "))
            if amt> int(cc[2]):
                print("Amount is more than due..... Please enter less
amount!!!!!!!!!!!!!!")
                continue
            else:
                course1[regno]=[cc[0],cc[1],cc[2]-amt]
                flag=False
                break
print(course1)
import pickle
file1=open("students.txt","bw")
pickle.dump(course1,file1)
file1.close()

```

PYTHON DIRECTORY AND FILE **MANAGEMENT**

The module called “os” need to be imported to perform directory and file management operations. Python directory and file management invloves the following,

- ✓ **Current directory (or) working directory**
- ✓ **Change the directory**
- ✓ **List the files of the directory**
- ✓ **Make a new directory**
- ✓ **Renaming a directory**
- ✓ **Removing a directory**

Program 8:

```
import os
# This would give location of the current directory
print("The current working directory is : \t ",os.getcwd())

# Create a directory "test1"
os.mkdir("test1")
print("Directory Created")

# Rename a file from abc.txt to xyz.txt
os.rename( "abc.txt", "xyz.txt" )
print("The file has been renamed successfully")

# Delete file test2.txt
os.remove("students.txt")
print("The file has been removed successfully")

# This would remove "/tmp/test" directory.
os.rmdir( "test" )
print("The directory requested to delete has been deleted")

# Changing a directory to "/home/newdir"
os.chdir("d:\Kit_final")
print("directory changed")
```

PYTHON EXCEPTION

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

These exceptions are processed using five statements. These are:

- Ø **try/except:** catch the error and recover from exceptions hoist by programmers or Python itself.
- Ø **try/finally:** Whether exception occurs or not, it automatically performs the clean-up action.
- Ø **assert:** triggers an exception conditionally in the code.
- Ø **raise:** manually triggers an exception in the code.
- Ø **with/as:** implement context managers in older versions of Python such as - Python 2.6 & Python 3.0.

Few important points to be remembered:

- Ø A single try statement can have multiple except statements. This is useful when the try block contains statements that may throw different types of exceptions.
- Ø You can also provide a generic except clause, which handles any exception.
- Ø After the except clause(s), you can include an else-clause. The code in the else-block executes if the code in the try: block does not raise an exception.
- Ø The else-block is a good place for code that does not need the try: block's protection.

List of Standard Exceptions –

No	Exception Name & Description
1	Exception Base class for all exceptions
2	StopIteration Raised when the next() method of an iterator does not point to any object.
3	SystemExit Raised by the sys.exit() function.
4	StandardError Base class for all built-in exceptions except StopIteration and SystemExit.

5	ArithmeticError Base class for all errors that occur for numeric calculation.
6	OverflowError Raised when a calculation exceeds maximum limit for a numeric type.
7	FloatingPointError Raised when a floating point calculation fails.
8	ZeroDivisionError Raised when division or modulo by zero takes place for all numeric types.
9	AssertionError Raised in case of failure of the Assert statement.
10	AttributeError Raised in case of failure of attribute reference or assignment.
11	EOFError Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
12	ImportError Raised when an import statement fails.
13	KeyboardInterrupt Raised when the user interrupts program execution, usually by pressing Ctrl+c.
14	LookupError Base class for all lookup errors.
15	IndexError Raised when an index is not found in a sequence.
16	KeyError Raised when the specified key is not found in the dictionary.
17	NameError Raised when an identifier is not found in the local or global namespace.
18	UnboundLocalError Raised when trying to access a local variable in a function or method but no value has been assigned to it.

19	EnvironmentError Base class for all exceptions that occur outside the Python environment.
20	IOError Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
21	IOError Raised for operating system-related errors.
22	SyntaxError Raised when there is an error in Python syntax.
23	IndentationError Raised when indentation is not specified properly.
24	SystemError Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
25	SystemExit Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
26	TypeError Raised when an operation or function is attempted that is invalid for the specified data type.
27	ValueError Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
28	RuntimeError Raised when a generated error does not fall into any category.
29	NotImplementedError Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

Program 1:

```
(a,b) = (6,0)
try:# simple use of try-except block for handling errors
    g = a/b
except ZeroDivisionError:
    print ("This is a DIVIDED BY ZERO error")
```

Program 2:

```
(a,b) = (6,0)
try:
    g = a/b
except ZeroDivisionError as s:
    k = s
    print (k)
```

Program 3:

```
def KelvinToFahrenheit(Temperature):
    assert (Temperature >= 0),"Colder than absolute zero!"
    return ((Temperature-273)*1.8)+32
print (KelvinToFahrenheit(273))
print (int(KelvinToFahrenheit(505.78)))
print (KelvinToFahrenheit(-5))
```

Program 4:

```
try:
    fh = open("testfile", "w")
    fh.write("This is my test file for exception handling!!")
except IOError:
    print ("Error: can't find file or read data")
else:
    print ("Written content in the file successfully")
```

Program 5:

```
try:
    fh = open("testfile", "r")
    fh.write("This is my test file for exception handling!!")
except IOError:
    print ("Error: can't find file or read data")
else:
    print ("Written content in the file successfully")
```

Program 6:

```
try:
    fh = open("testfile", "w")
    try:
        fh.write("This is my test file for exception handling!!")
    finally:
        print ("Going to close the file")
        fh.close()
except IOError:
    print ("Error: can't find file or read data")
```

Program 7:

```
try:
    N1,N2=eval(input("Enter 2 nos separated by commas : "))
    res= N1/N2
except ZeroDivisionError:
    print("Division by zero is the error caused!!!!!!")
    res=0
except SyntaxError:
    print("Commas are missing...")
except:
    print("Wrong input...")
else:
    print("No Exception")
finally:
    print("N1 is {} ,N2 is {}, Result is {}".format(N1,N2,res))
```

User Defined Exceptions:

Programmers may name their own exceptions by creating a new exception class. Exceptions need to be derived from the exception class, either directly or indirectly.

Program 8:

```
#defining a user defined error or exception as a class
class Error1(Exception):
    pass
class Error2(Exception):
    pass
while True:
    try:
```



```

i_num=int(input("Enter a number : "))
if i_num==0:
    raise Error1
if i_num<0:
    raise Error2
else:
    print("Valid Input")
    break
except Error1 :
    print("Value should not be zero ..... Please try with it again....")

except Error2:
    print("Value should be a positive Number..... Please try with it again....")

```

PYTHON NAMESPACE

Python Namespace:

- Namespace are the containers for mapping names to its objects.

Program 1:

```

a=2
b=2
print("a= ",id(a))
print("b= ",id(b))
print(id(2))
c="Namespace"
d="Scope"
e="csc"
print("id(c) = ",id(c))
print("id(d) = ",id(d))
print("id(e) = ",id(e))

```

Program 2:

```

a=100
print("id(a) = ",id(100))
print("id(a)= ",id(a))
a=a+1
print("id(a)= ",id(a))
print("id(101)= ",id(101))
b=100
print("id(b)= ",id(b))

```

Python Variable Scope:

Program 3:

```
def outer_function():
    a=20
    def inner_function():
        a=30
        print("a= ",a)
    inner_function()
    print("a= ",a)
a=10
outer_function()
print("a= ",a)
```

Program 4:

```
def outer_function():
    global a
    def inner_function():
        global a
        a=30
        print("a= ",a)
    inner_function()
    print("a= ",a)
a=10
outer_function()
print("a= ",a)
```

PYTHON CLASS AND OBJECT

- ü Python is an object oriented programming language.
- ü Almost everything in Python is an object, with its properties and methods.
- ü A Class is like an object constructor, or a "blueprint" for creating objects.
- ü A Class can also be called as a sketch or a plan.
- ü A Class cannot contain a static variable., But the class variable can act as a static variable.

Creating a class:

Program 1:

```
class MyClass:  
    x = 5  
print(MyClass)
```

Creating an object:

Program 2:

```
class MyClass: #Class name  
    """Hiii!!!!.....Am a new class!!!!"""# Comment Line or docstrings  
    x = 5      #Class attribute  
    def func(self):      #Userdefined function within a class..... Self  
        indicates current value to be considered...  
        x=100 #data attribute  
        print("Value of x inside (local declaration)the function is {}".format(x))  
  
print(MyClass.__doc__)  
p1 = MyClass()  
print("Value of x outside (global declaration)the function is {}".format(p1.x))  
p1.func()
```

Example for class and object

Program 3:

```
class BankAccount:  
    balance=0  
    def withdraw(self,amount):  
        self.balance-=amount  
        return self.balance  
  
    def deposit(self,amount):  
        self.balance+=amount
```

```
        return self.balance
def balanceCheck(self):
    print("\n New Balance is .... Rs.{0}".format(self.balance))

def bal_return(self):
    return self.balance
```

```
BT=BankAccount()
while True:
    print("1. Deposit\n")
    print("2. Withdraw\n")
    print("3. Balance\n")
    print("4. Exit\n")
    ch=int(input("Enter Your Choice...."))
    if ch==1:
        while True:
            amt=int(input("Enter amount to Deposit...."))
            if amt>0:
                BT.deposit(amt)
                break
            else:
                print("Enter valid amount")
                continue
    if ch==2:
        while True:
            amt=int(input("Enter amount to Withdraw...."))
            if BT.bal_return()>amt:
                BT.withdraw(amt)
                break
            else:
                BT.balanceCheck()
                print("\n Low Balance, Please enter less amount...")
                continue
    if ch==3:
        BT.balanceCheck()
    if ch==4:
        break
```

PYTHON CONSTRUCTORS

- ü A python constructor is used in instantiating the object of the class.
- ü The name of the constructor is always the same as `__init__()`.
- ü The constructor can accept the arguments when necessary to create the object for a class.
- ü Arguments are optional except the self.
- ü Python classes can be very easily executed only with the help of constructors.

The init() Function:

All classes have a function called `__init__()`, which is always executed when the class is being initiated. Use the `__init__()` function to assign values to object properties, or other operations that are necessary to do when the object is being created.

Program 1:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
p1 = Person("John", 36)
print("Name of the person is ", p1.name)
print("Age of the person is ", p1.age)
```

Program 2:

```
class student:
    def __init__(self, roll, name):
        self.r = roll
        self.n = name
        print("Student Roll No is : ", (self.r), "Student Name is : ", (self.n))

stud1 = student(1, "Alex")
stud2 = student(2, "Karlos")
print("Data successfully stored in variables")
```

Program 3:

```
class Employee:
    empCount = 0

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

```

Employee.empCount += 1

def displayCount(self):
    print ("Total Employee %d" % Employee.empCount)

def displayEmployee(self):
    print ("Name : ", self.name, ", Salary: ", self.salary)

emp1 = Employee("Hari", 20000)
emp2 = Employee("Giri", 10000)

emp1.displayEmployee()
emp2.displayEmployee()
print ("Total Employee %d" % Employee.empCount)

```

Python Constructor Overloading:

Python doesn't support method overloading or constructor overloading. It supports default arguments which can further easily accept different subsets of arguments.

Program 4:

```

class consover:
    def __init__(self,x=1,y=0):
        self.cx=x
        self.cy=y

    def callcons(self):
        print(" X value= %d, Y Value=%d" %(self.cx,self.cy))

b1=consover(10)
b2=consover(20,30)
b1.callcons()
b2.callcons()

```

Destroying Objects (Garbage Collection) (or) Destructor:

Python deletes unneeded objects (built-in types or class instances) automatically to free the memory space. The process by which Python periodically reclaims blocks of memory that no longer are in use is termed Garbage Collection.

Program 5:

```

class Point:
    def __init__( self, x=0, y=0):

```

```
    self.x = x
    self.y = y
def __del__(self):
    class_name = self.__class__.__name__
    print (class_name, "destroyed")
```

```
pt1 = Point()
pt2 = pt1
pt3 = pt1
print (id(pt1), id(pt2), id(pt3)) # prints the ids of the objects
del pt1
del pt2
del pt3
```

PYTHON INHERITANCE

- ü Python Inheritance is defined as a process of creating a class by deriving it from a preexisting class by listing the parent class in parentheses after the new class name.
- ü The already existing class is called as base class or parent class and the newly created class is called as derived class or child class.
- ü Derived class inherits the features of the base class and also it has some add-ons to its own class.
- ü The main advantage of inheritance is to maintain the re-usability of the code.

Types of Inheritance:

There are 3 different types of inheritance in python. They are,

- v Single Inheritance
- v Multiple Inheritance
- v Multilevel Inheritance

Single Inheritance:

Single inheritance is defined as the process of inheriting a single child class to a parent.

Program 1:

```
class Parent: # define parent class
    parentAttr = 100
    def __init__(self):
        print ("Calling parent constructor")

    def parentMethod(self):
        print ('Calling parent method')

    def setAttr(self, attr):
        Parent.parentAttr = attr

    def getAttr(self):
        print ("Parent attribute :", Parent.parentAttr)

class Child(Parent): # define child class
    def __init__(self):
        print ("Calling child constructor")
```



```
def childMethod(self):  
    print ('Calling child method')
```

```
c = Child() # instance of child  
c.childMethod() # child calls its method  
c.parentMethod() # calls parent's method  
c.setAttr(200) # again call parent's method  
c.getAttr() # again call parent's method
```

Program 2:

```
class Person:  
    def __init__(self,no,name):  
        self.Empno=no  
        self.Ename=name  
  
    def dispdata(self):  
        print("Employee No is ",self.Empno)  
        print("Employee Name is ",self.Ename)  
  
class Employee(Person):  
    def callme(self):  
        print("I can use the attributes of person")  
  
perobj=Person(101,"RamKumar")  
empobj=Employee(102,"Suresh")  
print("Employee Details ")  
print("-"*25)  
perobj.dispdata()  
empobj.dispdata()
```

Program 3:

```
class Polygon:  
    def __init__(self, no_of_sides):  
        self.n=no_of_sides  
        self.sides=[0 for i in range(no_of_sides)]  
  
    def inputSides(self):
```

```

        self.sides=[float(input("Enter side " +str(i+1) + " : ")) for i in range (self.n)]

    def dispSides(self):
        for i in range(self.n):
            print("Side ",i+1, "is ",self.sides[i])

class Triangle(Polygon):
    def __init__(self):
        Polygon.__init__(self,3)

    def findArea(self):
        a,b,c=self.sides
        s=(a+b+c)/2
        area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
        print("The area of the triangle is %0.2f "%area)

t=Triangle()
t.inputSides()
t.dispSides()
t.findArea()

```

Multiple Inheritance:

Multiple inheritance is defined as the process of inheriting a single child class to more than a single parent class.

Program 4:

```

class Student(object):
    Total=0; Average=0.0; Result=" "
    def __init__(self,no,na):
        self.stdno=no
        self.stdname=na

    def stdFun(self):
        print("Student No is : ",self.stdno, "\n Student Name : ",self.stdname)

class Test1(object):
    def __init__(self,m1,m2):
        self.p1=m1
        self.p2=m2

```

```
def Markdisp1(self):  
    print("C Language : ",self.p1)  
    print("C++ : ",self.p2)
```

```
class Test2(object):  
    def __init__(self,m3,m4):  
        self.p3=m3  
        self.p4=m4
```

```
def Markdisp2(self):  
    print("Java : ",self.p3)  
    print("Python : ",self.p4)
```

```
class Result(Student,Test1,Test2):  
    def __init__(self,no,na,m1,m2,m3,m4):  
        Student.__init__(self,no,na)  
        Test1.__init__(self,m1,m2)  
        Test2.__init__(self,m3,m4)
```

```
def CallFun(self):  
    self.Total= self.p1+self.p2+self.p3+self.p4  
    self.Average=self.Total/4  
    if self.p1>=40 and self.p2>=40 and self.p3>=40 and self.p4>=40:  
        self.Result ="Pass"  
  
    else:  
        self.Result ="Fail"
```

```
def ResultDisplay(self):  
    print("Total : ",self.Total)  
    print("Average : ",self.Average)  
    print("Result : ",self.Result)
```

```
s1=Result(101,"Raj",56,67,78,90)  
s1.CallFun()  
print("-" *25)
```

```
s1.stdFun()
print("-" *25)
s1.Markdisp1()
s1.Markdisp2()
print("-" *25)
s1.ResultDisplay()
print("-" *25)
```

Multi Level Inheritance:

Multi Level inheritance is defined as the process of inheriting a many child class from many single parent class.

Program 5:

```
#main class(parent class)
class Student:
    def getStudentDetails(self):
        self.name=input("Name : ")
        self.age=int(input("Age : "))
        self.gender=input("Gender : ")

#first subclass
class Test(Student):
    def getMarks(self):
        self.stuClass = input("Diploma Name : ")
        print("Enter the marks of the respective subjects")
        self.c=int(input("C Language : "))
        self.cplusplus=int(input("C++ : "))
        self.java=int(input("Java : "))
        self.python=int(input("Python : "))

#second subclass
class Marks(Test):
    def display(self):
        print("\n\n Name: ",self.name)
        print("Age : ",self.age)
        print("Gender : ",self.gender)
        print("Diploma Name : ",self.stuClass)
        print("Total Marks : {} /400".format(self.c +self.cplusplus +self.java
+self.python))
```

```
#creating object for the class  
m1=Marks()  
m1.getStudentDetails()  
m1.getMarks()  
m1.display()
```

Built-in class attributes:

- `__dict__`: This attribute is a dictionary that contains class's-namespace.
- `__doc__`: Used for class documentation string.
- `__name__`: used as class-name.
- `__module__`: used to define module name for the class in which it is defined. In interactive mode it is `__main__`.
- `__bases__`: An empty tuple containing the base-class.

Program :

```
class Employee:
    empCount = 0

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.empCount += 1

    def displayCount(self):
        print ("Total Employee %d" % Employee.empCount)

    def displayEmployee(self):
        print ("Name : ", self.name, ", Salary: ", self.salary)

print ("\nEmployee.__doc__:", Employee.__doc__)
print ("\nEmployee.__name__:", Employee.__name__)
print ("\nEmployee.__module__:", Employee.__module__)
print ("\nEmployee.__bases__:", Employee.__bases__)
print ("\nEmployee.__dict__:", Employee.__dict__)
```

Program :

#Use the words mysillyobject and abc instead of self:

```
class Person:
    def __init__(mysillyobject, name):
        mysillyobject.n = name

    def myfunc(abc):
        print("My name is " + abc.n)

p1 = Person("John")
p1.myfunc()
```

Polymorphism:

Polymorphism is defined as the process of creating a structure that can take many forms of objects. Here poly represents “many” and morphism represents “forms”

Program:

```
class Parent: # define parent class
    def myMethod(self):
        print ('Calling parent method')

class Child(Parent): # define child class
    def myMethod(self):
        print ('Calling child method')

c = Child() # instance of child
c.myMethod() # child calls overridden method
```

Program:

```
class Branch1:
    def display(self):
        print("C cannot be implemented using OOPS")

class Branch2:
    def display(self):
        print("C++ can be implemented using OOPS")

def myfunction(l):
    l.display()
B1 = Branch1()
B2 = Branch2()
myfunction(B1)
myfunction(B2)
```

Python operator overloading:

```
class oprover:
    def __init__(self, a, b):
        self.a = a
        self.b = b
    def __str__(self):
        return ('Vector (%d, %d)' % (self.a, self.b))

    def __add__(self, other):
        return oprover(self.a + other.a, self.b + other.b)
```

```
v1 = oprover(2,10)
v2 = oprover(5,-2)
print (v1 + v2)
```


Types of Attributes:

The non-method data stored by an object are called as attributes. There are 2 different types of attributes

- Ø Class attribute
- Ø Data attribute

Class attributes:

- Ø Class attributes are owned by a class as a whole, all instances share the same value(shares one copy of class attribute).
- Ø When one instance is changed all other instances also changes.
- Ø Class attribute is defined within a class and outside any method.
- Ø Class attribute is represented as self.__class__. (anyvariablename)
- Ø It is also called as “static” variables.

Data attributes:

- Ø Data variables are owned by a particular instance of a class. Each instance has its own value for it.
- Ø Data attribute is represented as __init__().

Program 3:(Example involving class and data attribute)

```
class attribute:
```

```
    num=10
    def __init__(self,x):
        self.num=x
    def count_fun(self):
        self.__class__.num+=1
```

```
N1=attribute(15)
```

```
N2=attribute(20)
```

```
print("\n N1 Object: Data Attribute 'num' value is ",N1.num)
```

```
print("\n N1 Object: Class Attribute 'num' value is ",N1.__class__.num)
```

```
print("\n N2 Object: Data Attribute 'num' value is ",N2.num)
```

```
print("\n N2 Object: Class Attribute 'num' value is ",N2.__class__.num)
```

```
N1.count_fun()
```

```
N2.count_fun()
```

```
print("\n After 2 times the count function execution is ")
```

```
print("\n N1 Object: Data Attribute 'num' value is ",N1.num)
```

```
print("\n N1 Object: Class Attribute 'num' value is ",N1.__class__.num)
```

```
print("\n N2 Object: Data Attribute 'num' value is ",N2.num)
```

```
print("\n N2 Object: Class Attribute 'num' value is ",N2.__class__.num)
```

```
#Printing class variable using class name
print("\n Data attribute num value ",attribute.num)
```

Program 4:

```
class student:
    #class variable
    branch="T.Nagar"

    #The init method or the constructor
    def __init__(self,rollno,name):
        #instance variable
        self.sno=rollno
        self.sname=name
    def dispdata(self):
        #it can be accessed by self or class name
        print("Branch Name is ", student.branch)
        print("Enrollment no is ", self.sno)
        print("Student Name is ",self.sname)
        print("\n\n")

#object creation
s1=student('PR25001',"Rakshan")
s2=student('PR25002',"Rakshitha")
s1.dispdata()
s2.dispdata()

print(s1.branch)
print(s2.branch)
print(s1.sno)
print(s2.sno)
#class variables can be accessed using class name also
print(student.branch)
```

Program 5:

```
class student:
    #class variable
    branch = 'T.Nagar'
    #The init method or the constructor
    def __init__(self,rollno,name,branch):
```

```
#instance variable
self.sno=rollno
self.sname=name
self.branch=branch
#The variable self.branch is a instance & branch is a formal parameter
```

```
def dispdata(self):
    print("Branch Name is ",self.branch)
    print("Enrollment No is ",self.sno)
    print("Name is ",self.sname)
```

```
#Object creation
s1=student('PR25001','Rakesh','Vandalur')
s2=student('PR25002','Ramesh','Mylapore')
s1.dispdata()
s2.dispdata()
print(s1.branch)
print(s2.branch)
print(s1.sno)
print(s2.sno)
```

```
#class variable can be accessed using class variable also
print(student.branch)
```

Program 6:(Program involving private method and private variable)

```
class pvariable:
    __a=0
    __b=0
    c=0
    def __funcall(self,m,n):
        self.__a=m
        self.__b=n
    def sum(self):
        result=self.__a+self.__b+self.c
        return (result)
    def maincall(self,m,n):
        self.__funcall(m,n)
```

```
o1=pvariable()
o1.c=40
```

```
o1.maincall(100,200)
print("Sum value is %d" % o1.sum())
```

PYTHON MULTITHREADING

- v A thread has a beginning, an execution sequence, and a conclusion.
- v It is the execution of a tiny sequence of program instruction that can be managed independently and is a distinctive part of operating system.
- v It has an instruction pointer that keeps track of where within its context it is currently running.
- v Threads sometimes called light-weight processes and they do not require much memory overhead; they are cheaper than processes.
- v It can temporarily be put on hold (also known as sleeping) while other threads are running - this is called yielding.
- v Multiple threads within a process share the same data space with the main thread and can therefore share information or communicate with each other more easily than if they were separate processes.

Uses of Thread:

Thread plays a significant role in application programming. All the GUI programs and web servers are threaded together. The main reasons for using threads are:

- **Parallel Computation:** If any user has multiprocessor machine then the thread can allow doing parallel processing with the goal of increase in processing speed.
- **Standardization:** It became a standard approach for all programming languages as it increases programming speed.
- **Parallel I/O:** When we talk about the speed of input & output, it is comparatively slow in CPU. By placing each i/o operations in multiple individual threads, programmers can make use of operations done in parallel with each other & with the computation speed.

Creating Thread Using Threading Module:

To implement a new thread using the threading module, we must

- Ø Define a new subclass of the *Thread* class.
- Ø Override the `__init__(self [,args])` method to add additional arguments.
- Ø Then, override the `run(self [,args])` method to implement what the thread should do when started.

Once we have created the new *Thread* subclass, we can create an instance of it and then start a new thread by invoking the `start()`, which in turn calls `run()` method.

Threading Module and Methods:

The threading module exposes all the methods of the thread module and provides some additional methods:

- Ø **threading.activeCount():** Returns the number of thread objects that are active.
- Ø **threading.currentThread():** Returns the number of thread objects in the caller's thread control.
- Ø **threading.enumerate():** Returns a list of all thread objects that are currently active.

In addition to the methods, the threading module has the *Thread* class that implements threading. The methods provided by the *Thread* class are

- Ø **run():** It acts as the entry of the thread
- Ø **start():** is used for starting the thread by calling the `run()`
- Ø **isAlive():** is used to verify whether the thread is still executing or not
- Ø **getName():** is used for returning the name of a thread
- Ø **setName():** is used to set the name of the thread
- Ø **join([time]):** The `join()` waits for threads to terminate.

Program 1:

```
import threading
def coder(number):
    print ("\nCoders: %s" %number)

for k in range(5):
    t = threading.Thread(target=coder, args=(k,))
    t.start()
```

Program 2:

```
import time
import threading
from threading import Thread

def sleepMe(i):
    print("Thread %i going to sleep for 5 seconds." % i)
    time.sleep(5)
    print("Thread %i is awake now." % i)

for i in range(10):
    th = Thread(target=sleepMe, args=(i, ))
    th.start()
    print("Current Thread count: %i." % threading.active_count())
```

Program 3:

```
import threading
import time
def loop1_10():
    for i in range(1, 11):
        time.sleep(5)
        print(i)

threading.Thread(target=loop1_10).start()
```

Program 4:

```
import threading
import time

class MyThread(threading.Thread):
    def run(self):
        print("{} started!".format(self.getName()))        # "Thread-x started!"
        time.sleep(1)                                       # Pretend to work for a second
        print("{} finished!".format(self.getName()))        # "Thread-x finished!"

if __name__ == '__main__':
    for x in range(4):                                     # Four times...
        mythread = MyThread(name = "Thread-{}".format(x + 1)) # ...Instantiate
                                                                # a thread and pass a unique ID to it
        mythread.start()                                    # ...Start the thread
        time.sleep(.8)  # ...Wait 0.8 seconds before starting another
```

Program 5:

```
import threading
import datetime

class ThreadClass(threading.Thread):
    def run(self):
        now_datetime = datetime.datetime.now()
        print( "\n%s says Hello World at time: %s" %(self.getName(),
now_datetime))

for i in range(2):
    t = ThreadClass()
    t.start()
```

Program 6:

```
import threading
import time

exitFlag = 0

class myThread (threading.Thread):
    def __init__(self, threadID, name, counter):
        threading.Thread.__init__(self)
        self.threadID = threadID
        self.name = name
        self.counter = counter
    def run(self):
        print("Starting " + self.name)
        print_time(self.name, 5, self.counter)
        print ("Exiting " + self.name)

def print_time(threadName, counter, delay):
    while counter:
        if exitFlag:
            threadName.exit()
        time.sleep(delay)
        print ("%s: %s" % (threadName, time.ctime(time.time())))
        counter -= 1

# Create new threads
thread1 = myThread(1, "Thread-1", 1)
thread2 = myThread(2, "Thread-2", 2)
```



```
# Start new Threads
thread1.start()
thread2.start()

print ("Exiting Main Thread")
```

Program 7:

```
import time
import threading
from threading import Thread

def sleepMe(i):
    print("\nThread %s going to sleep for 5 seconds." %
threading.current_thread())
    time.sleep(5)
    print("Thread %s is awake now.\n" % threading.current_thread())

#Creating only four threads for now
for i in range(4):
    th = Thread(target=sleepMe, args=(i, ))
    th.start()
    print("\nCurrent Thread count: %i." % threading.active_count())
```

PYTHON GUI PROGRAMMING(TKINTER)

Tkinter:

- Ø TKinter was invented by Fredrik Lundh.
- Ø Tkinter is the easiest among all to get started with GUI programming and it remains as the most commonly used toolkit.
- Ø It is Python's standard GUI (Graphical User Interface) package.
- Ø Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit.
- Ø Creating a GUI application using Tkinter,
 - ü Import the Tkinter module.
 - ü Create the GUI application main window.
 - ü Add one or more of the widgets to the GUI application.
 - ü Enter the main event loop to take action against each event triggered by the user.

Sample program to create the screen:

```
from tkinter import *  
top = Tk()  
top.mainloop()
```

Tkinter Widgets:

<u>Operator</u>	<u>Description</u>
Button	The Button widget is used to display buttons.
Canvas	The Canvas widget is used to draw shapes, such as lines, ovals, polygons and rectangles.
Checkbutton	The Checkbutton widget is used to display a number of options as checkboxes. The user can select multiple options at a time.
Entry	The Entry widget is used to display a single-line text field for accepting values from a user.
Frame	The Frame widget is used as a container widget to organize other widgets.
Label	The Label widget is used to provide a single-line caption for other widgets. It can also contain images.
Listbox	The Listbox widget is used to provide a list of options to a user.
Menubutton	The Menubutton widget is used to display menus.

Menu	The Menu widget is used to provide various commands to a user. These commands are contained inside Menubutton.
Message	The Message widget is used to display multiline text fields for accepting values from a user.
Radiobutton	The Radiobutton widget is used to display a number of options as radio buttons. The user can select only one option at a time.
Scale	The Scale widget is used to provide a slider widget.
Scrollbar	The Scrollbar widget is used to add scrolling capability to various widgets, such as list boxes.
Text	The Text widget is used to display text in multiple lines.
Toplevel	The Toplevel widget is used to provide a separate window container.
Spinbox	The Spinbox widget is a variant of the standard Tkinter Entry widget, which can be used to select from a fixed number of values.
Panedwindow	A PanedWindow is a container widget that may contain any number of panes, arranged horizontally or vertically.
LabelFrame	A labelframe is a simple container widget. Its primary purpose is to act as a spacer or container for complex window layouts.
tkMessageBox	This module is used to display message boxes.

Program 1: (Label,Entry)

```
from tkinter import *
top = Tk()
```

```
L1 = Label(top, text="User Name")
L2 = Label(top, text="Password")
```

```
E1 = Entry(top, bd =20, bg="green",fg="white",font=("Times new roman",30))
E2 = Entry(top, bd =50, bg="pink",)
```

```
E1.pack(side = LEFT)
E2.pack(side = RIGHT)
L1.pack( side = LEFT)
L2.pack(side = RIGHT)
```

```
top.mainloop()
```

Program 2: (LabelFrame, Entry, StringVar)

```
from tkinter import *

root = Tk()
Label(text="Enter any 1 fruit name from the below given Frame!!!").pack()
entryContent = StringVar()
Entry(root, textvariable=entryContent).pack()
frame = LabelFrame(root, text="Fruits") # text is optional
frame.pack()
Label(frame, text="Apple").pack()
Label(frame, text="Orange").pack()
Label(frame, text="Mango").pack()
Label(frame, text="Pineapple").pack()
Label(frame, text="Grapes").pack()

mainloop()
print (entryContent.get())
```

Program 3: (RadioButton, StringVar, Button)

```
from tkinter import *

master = Tk()
choices = [("Apple", "a"), ("Orange", "o"), ("Pear", "p")]
defaultChoice = "a"
userchoice = StringVar()
userchoice.set(defaultChoice)

def cancelAction():
    userchoice.set("");master.quit()

Label(master, text="Choose a fruit:").pack()
for t, k in choices:
    Radiobutton(master, text=t, variable=userchoice, value=k).pack(anchor=W)

Button(master, text="OK", command=master.quit).pack(side=LEFT, ipadx=100,
ipady = 20)
Button(master, text="Cancel", command=cancelAction).pack(side=RIGHT,
ipadx=10, ipady = 200)
```

```
mainloop()

if userchoice.get() :
    print (userchoice.get()) # "a", or "o", or "p"
else:
    print ("Choice canceled.")
```

Program 4: (Canvas)

```
import tkinter
top = tkinter.Tk()

C = tkinter.Canvas(top, bg="blue", height=250, width=300)
coord = 10, 50, 240, 210
arc = C.create_arc(coord, start=0, extent=150, fill="red")

C.pack()
top.mainloop()
```

PYTHON GUI PROGRAMMING(TKINTER)

Program 5: (ListBox)

```
from tkinter import *
master = Tk()
choices = ["DELL", "HP", "TOSHIBA", "SONY", "ACER"]
canceled = BooleanVar()

def cancelAction(): canceled.set(True);

Label(master, text="Choose a Laptop Company:").pack()
listbox = Listbox(master, selectmode=EXTENDED) # Multiple options can be
chosen

for text in choices:
    listbox.insert(END, text)

listbox.pack()
Button(master, text="OK", command=master.quit).pack(side=LEFT, padx=10)
Button(master, text="Cancel", command=cancelAction).pack(side=RIGHT,
ipadx=10)
mainloop()

if not canceled.get():
    print (listbox.curselection()) # A tuple of choice indices starting with 0
    # The above is a tuple even if selectmode=SINGLE
    if "0" in listbox.curselection(): print ("DELL is selected.")
    if "1" in listbox.curselection(): print ("HP is selected.")
    if "2" in listbox.curselection(): print ("TOSHIBA is selected." )
    if "3" in listbox.curselection(): print ("SONY is selected" )
    if "4" in listbox.curselection(): print ("ACER is selected." )
else:
    print ("Choice canceled.")
```

Program 6: (Checkbox)

```
from tkinter import *
```

```

import tkinter

top = tkinter.Tk()
CheckVar1 = IntVar()
CheckVar2 = IntVar()
CheckVar3 = IntVar()
C1 = Checkbutton(top, text = "Chocolate", variable = CheckVar1, \
                 onvalue = 1, offvalue = 0, height=5, width = 20)
C2 = Checkbutton(top, text = "Blackcurrent", variable = CheckVar2, \
                 onvalue = 1, offvalue = 0, height=5, width = 20)
C3 = Checkbutton(top, text = "Strawberry", variable = CheckVar3, \
                 onvalue = 1, offvalue = 0, height=5, width = 20)
C1.pack()
C2.pack()
C3.pack()

top.mainloop()
CheckVar1.get()
CheckVar2.get()
CheckVar3.get()

if CheckVar1.get() == 1 and CheckVar2.get() == 1 and CheckVar3.get()==1:
    print("All the ice-cream floavours are selected")
elif CheckVar1.get() == 1 and CheckVar2.get() == 1 and CheckVar3.get()==0 :
    print("Chocolate and Blackcurrent are selected")
elif CheckVar1.get() == 1 and CheckVar2.get() == 0 and CheckVar3.get()==1 :
    print("Chocolate and Strawberry are selected")
elif CheckVar1.get() == 1 and CheckVar2.get() == 0 and CheckVar3.get()==0 :
    print("Chocolate is selected")
elif CheckVar1.get() == 0 and CheckVar2.get() == 1 and CheckVar3.get()==1 :
    print("Blackcurrent and Strawberry are selected")
elif CheckVar1.get() == 0 and CheckVar2.get() == 1 and CheckVar3.get()==0 :
    print("Blackcurrent is selected")
elif CheckVar1.get() == 0 and CheckVar2.get() == 0 and CheckVar3.get()==1 :
    print("Strawberry is selected")
elif CheckVar1.get() == 0 and CheckVar2.get() == 0 and CheckVar3.get()==0 :
    print("None of the ice-cream flavours are selected")

```

Program 7: (Labelframe & Spinbox)

```

from tkinter import *

```

```
root = Tk()

labelframe = LabelFrame(root, text="This is a LabelFrame")
labelframe.pack(fill="both", expand="yes")
left = Label(labelframe, text="Inside the LabelFrame")
w = Spinbox(labelframe, from_=0, to=10)
left.pack()
w.pack(side=LEFT)

root.mainloop()
print("The labelframe and spinbox has been applied in the root canvas!!!!!!!!!!!!!!")
```

Program 8: (Text & Message)

```
from tkinter import *

def onclick():
    pass

root = Tk()
text = Text(root)
text.insert(INSERT, "Hello.....")
text.insert(END, "Bye Bye.....")
text.pack()

text.tag_add("here", "1.0", "1.4")
text.tag_add("start", "1.8", "1.13")
text.tag_config("here", background="yellow", foreground="blue")
text.tag_config("start", background="black", foreground="green")
Message(text="Lazy brown fox jumped over the black dog. " * 5,
width=100).pack() # width is optional
root.mainloop()
```

Program 9: (OptionMenu)

```
from tkinter import *

root = Tk()
options = ["Red", "Blue", "Green", "Violet", "Orange"]
```



```
selectedOption = StringVar()
selectedOption.set("Apple") # Default
OptionMenu(root, selectedOption, *options).pack()
mainloop()
print (selectedOption.get()) # The text in the options list
```

PYTHON GUI PROGRAMMING(TKINTER)

Program 10: (RadioButton)

```
from tkinter import *
```

```
def sel():
```

```
    selection = "You selected the option " + str(var.get())
```

```
    label.config(text = selection)
```

```
root = Tk()
```

```
var = IntVar()
```

```
R1 = Radiobutton(root, text="Option 1", variable=var, value=1,  
                  command=sel)
```

```
R1.pack( anchor = W )
```

```
R2 = Radiobutton(root, text="Option 2", variable=var, value=2,  
                  command=sel)
```

```
R2.pack( anchor = W )
```

```
R3 = Radiobutton(root, text="Option 3", variable=var, value=3,  
                  command=sel)
```

```
R3.pack( anchor = W)
```

```
label = Label(root)
```

```
label.pack()
```

```
root.mainloop()
```

Program 11: (ScrollBar)

```
from tkinter import *
```

```
root = Tk()
```

```
scrollbar = Scrollbar(root)
```

```
scrollbar.pack( side = RIGHT, fill = Y )
```

```
mylist = Listbox(root, yscrollcommand = scrollbar.set )
```

```
for line in range(100):
```

```
mylist.insert(END, "This is line number " + str(line))

mylist.pack( side = LEFT, fill = BOTH )
scrollbar.config( command = mylist.yview )
```

Program 12: (Scale)

```
from tkinter import *
master = Tk()
w = Scale(master,from_=0, to=100)
w.pack()
w=Scale(master,from_=0, to=100, orient= HORIZONTAL)
w.pack()
master.mainloop()
```

Program 13: (PanedWindow)

```
from tkinter import *

m1 = PanedWindow()
m1.pack(fill=BOTH, expand=1)
left = Label(m1, text="left pane")
m1.add(left)
m2 = PanedWindow(m1, orient=VERTICAL)
m1.add(m2)
top = Label(m2, text="top pane")
m2.add(top)
bottom = Label(m2, text="bottom pane")
m2.add(bottom)

mainloop()
```

Program 14: (Menu)

```
from tkinter import *
root = Tk()
def mycommand(): print ("Chosen.")
menubar = Menu(root)
```

```
menu1 = Menu(menubar, tearoff=0)
menu1.add_command(label="New", command=mycommand)
menu1.add_command(label="Clone", command=mycommand)
menu1.add_separator()
menu1.add_command(label="Exit", command=root.quit)
menubar.add_cascade(label="Project", menu=menu1)

menu2 = Menu(menubar, tearoff=0)
menu2.add_command(label="Oval", command=mycommand)
menu2.add_command(label="Rectangle", command=mycommand)
menubar.add_cascade(label="Shapes", menu=menu2)

root.config(menu=menubar)

mainloop()
```

Program 15: (MenuBar& Toplevel)

```
from tkinter import *
def donothing():
    filewin = Toplevel(root)
    button = Button(filewin, text="Do nothing button")
    button.pack()

root = Tk()
menubar = Menu(root)
filemenu = Menu(menubar, tearoff=0)
filemenu.add_command(label="New", command=donothing)
filemenu.add_command(label="Open", command=donothing)
filemenu.add_command(label="Save", command=donothing)
filemenu.add_command(label="Save as...", command=donothing)
filemenu.add_command(label="Close", command=donothing)

filemenu.add_separator()

filemenu.add_command(label="Exit", command=root.quit)
menubar.add_cascade(label="File", menu=filemenu)
editmenu = Menu(menubar, tearoff=0)
editmenu.add_command(label="Undo", command=donothing)
```

```
editmenu.add_separator()
```

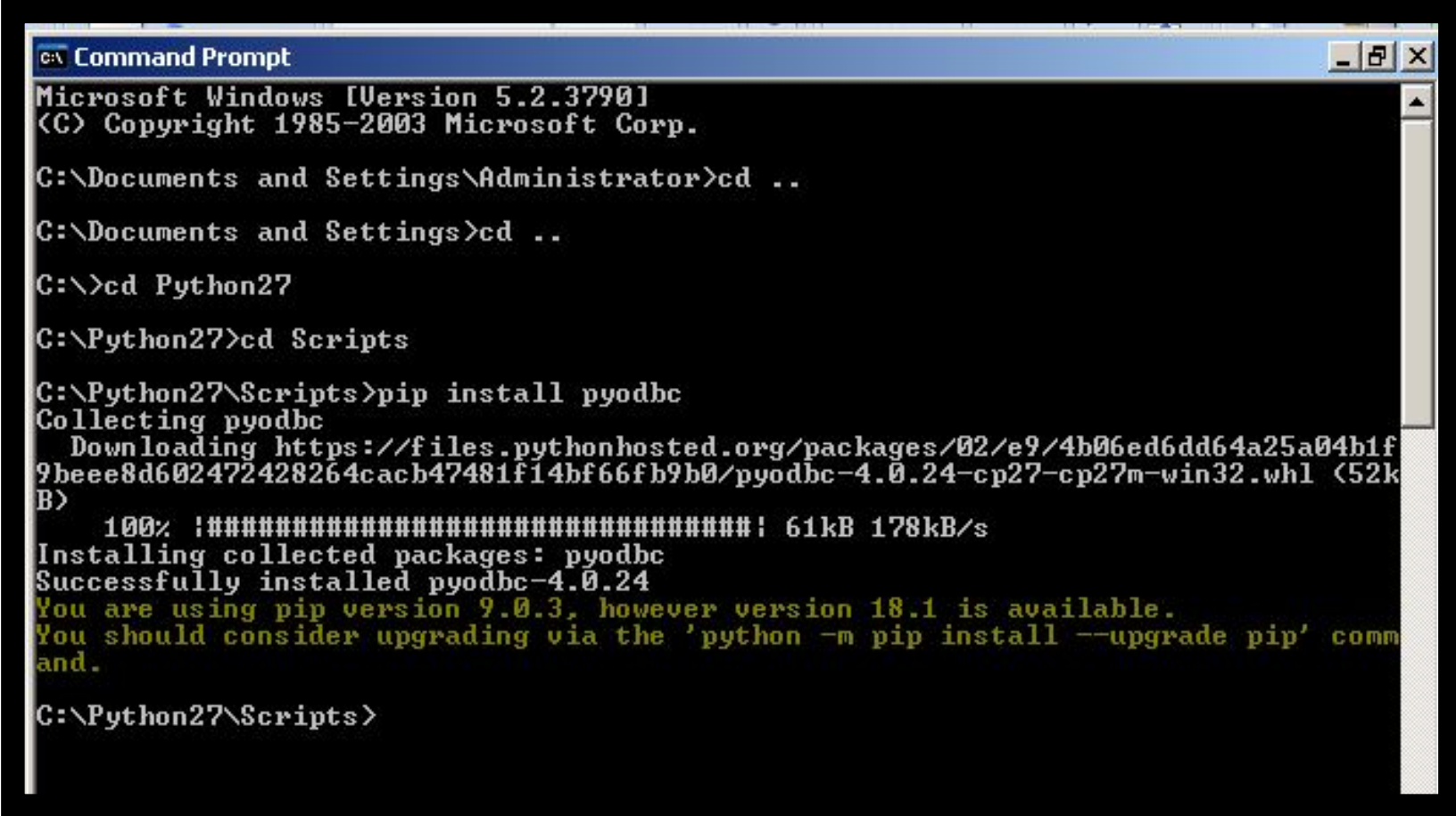
```
editmenu.add_command(label="Cut", command=donothing)  
editmenu.add_command(label="Copy", command=donothing)  
editmenu.add_command(label="Paste", command=donothing)  
editmenu.add_command(label="Delete", command=donothing)  
editmenu.add_command(label="Select All", command=donothing)
```

```
menubar.add_cascade(label="Edit", menu=editmenu)  
helpmenu = Menu(menubar, tearoff=0)  
helpmenu.add_command(label="Help Index", command=donothing)  
helpmenu.add_command(label="About...", command=donothing)  
menubar.add_cascade(label="Help", menu=helpmenu)
```

```
root.config(menu=menubar)  
root.mainloop()
```

Python DataBase Connectivity Using Ms-Access

Method to install Pip



```
C:\ Command Prompt
Microsoft Windows [Version 5.2.3790]
(C) Copyright 1985-2003 Microsoft Corp.

C:\Documents and Settings\Administrator>cd ..
C:\Documents and Settings>cd ..
C:\>cd Python27
C:\Python27>cd Scripts
C:\Python27\Scripts>pip install pyodbc
Collecting pyodbc
  Downloading https://files.pythonhosted.org/packages/02/e9/4b06ed6dd64a25a04b1f9beee8d602472428264cacb47481f14bf66fb9b0/pyodbc-4.0.24-cp27-cp27m-win32.whl (52kB)
    100% |#####| 61kB 178kB/s
Installing collected packages: pyodbc
Successfully installed pyodbc-4.0.24
You are using pip version 9.0.3, however version 18.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

C:\Python27\Scripts>
```

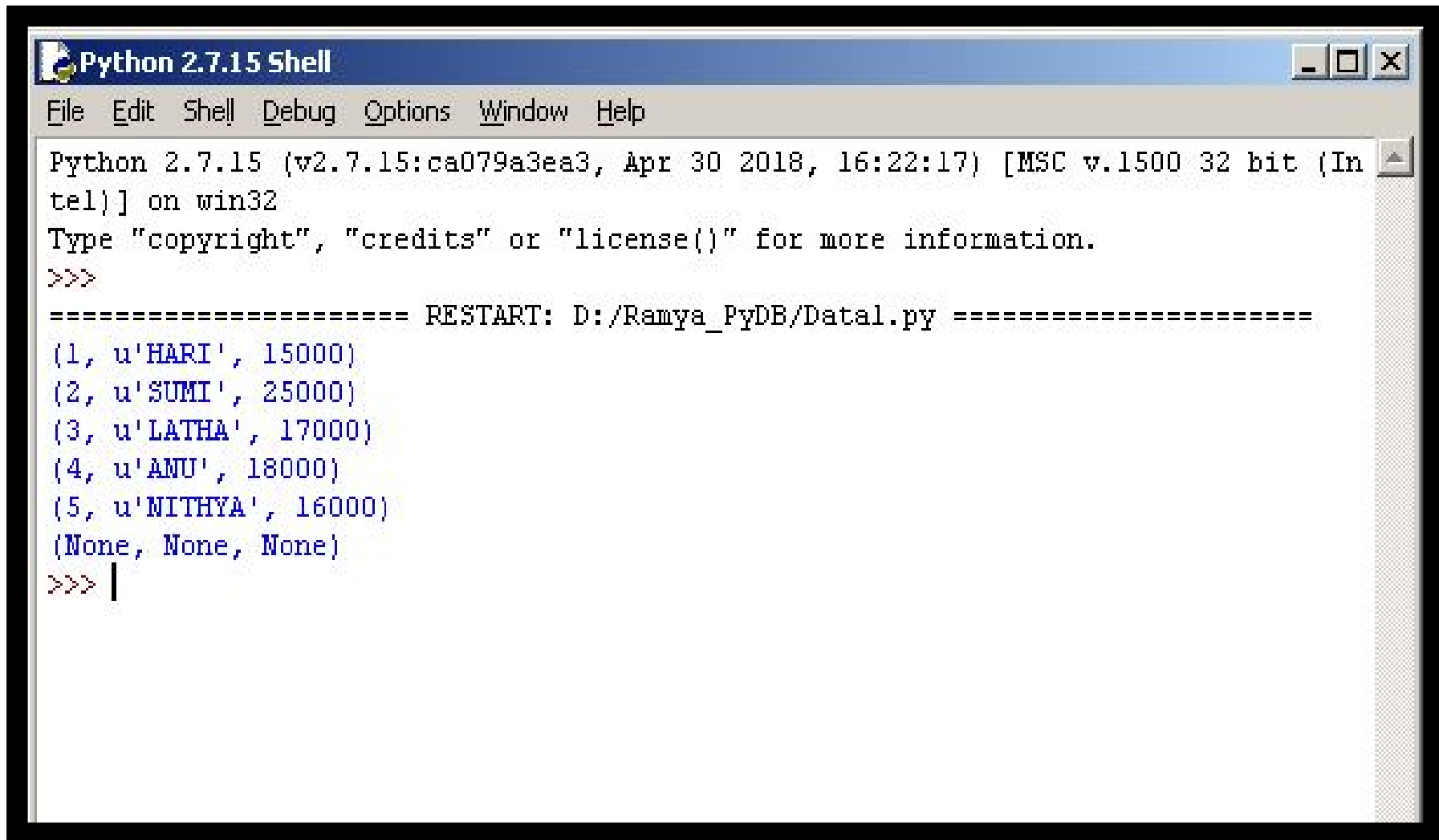
Program to view the data from Ms-Access to Python

```
import pyodbc
```

```
conn = pyodbc.connect(r'Driver={Microsoft Access  
Driver (*.mdb,  
*.accdb)};DBQ=d:\Ramya_PyDB\Mydatabase.accdb;')  
cursor = conn.cursor()  
cursor.execute('select * from mytable')
```

```
for row in cursor.fetchall():  
    print (row)
```


Output



```
Python 2.7.15 Shell
File Edit Shell Debug Options Window Help
Python 2.7.15 (v2.7.15:ca079a3ea3, Apr 30 2018, 16:22:17) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:/Ramya_PyDB/Datal.py =====
(1, u'HARI', 15000)
(2, u'SUMI', 25000)
(3, u'LATHA', 17000)
(4, u'ANU', 18000)
(5, u'NITHYA', 16000)
(None, None, None)
>>> |
```

Program to insert the multiple records into Ms-Access through Python

```
import pyodbc
```

```
conn = pyodbc.connect(r'Driver={Microsoft Access Driver (*.mdb, *.accdb)};DBQ=d:\Ramya_PyDB\Mydatabase.accdb;')
```

```
cursor = conn.cursor()
```

```
cursor.execute("INSERT INTO mytable (no, name, salary)  
VALUES('15', 'Jagan',15000)")
```

```
cursor.execute("INSERT INTO mytable (no,name,salary)  
VALUES('121', 'Mohan',26000)")
```

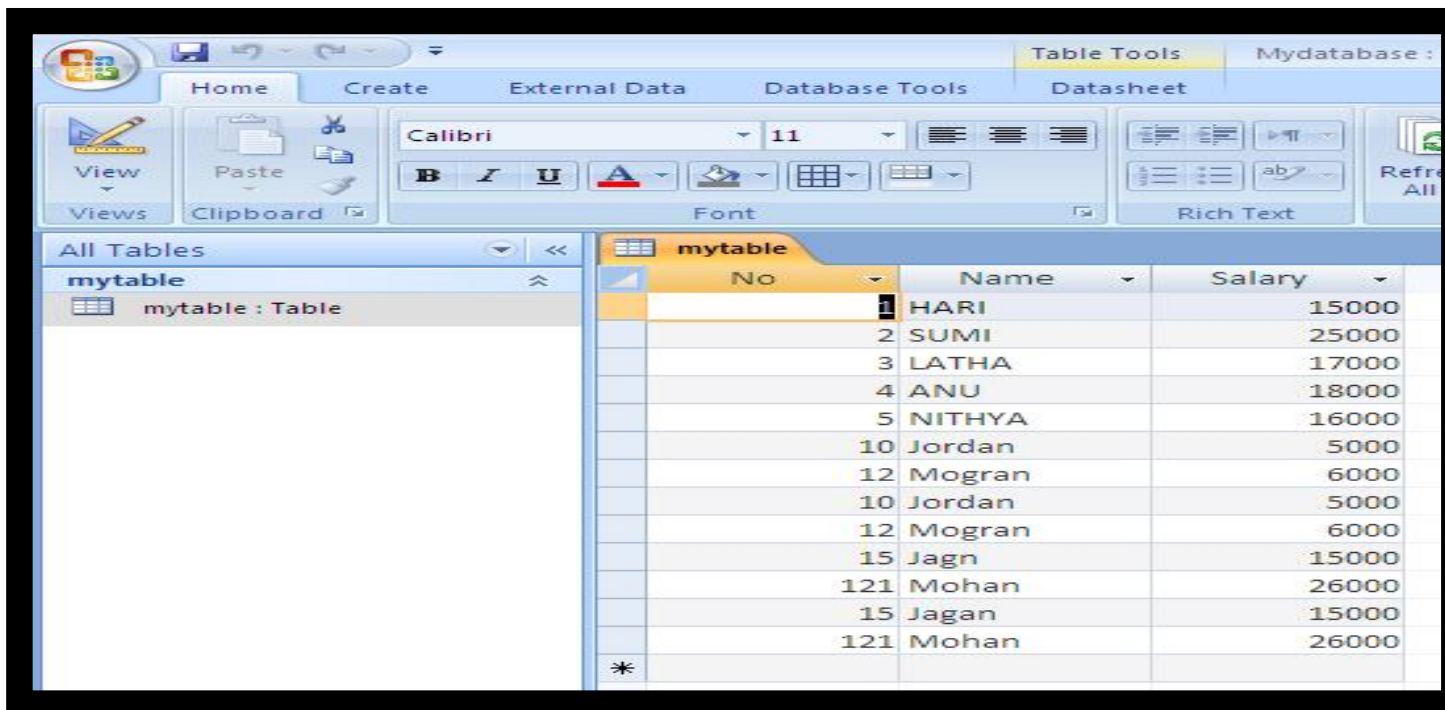
```
conn.commit()
```

```
cursor.execute('select * from mytable')
```

```
for row in cursor.fetchall():  
    print (row)
```

Output

```
===== RESTART: D:/Ramya_PyDB/Data1.py =====  
=====  
(1, u'HARI', 15000)  
(2, u'SUMI', 25000)  
(3, u'LATHA', 17000)  
(4, u'ANU', 18000)  
(5, u'NITHYA', 16000)  
(10, u'Jordan', 5000)  
(12, u'Mogran', 6000)  
(10, u'Jordan', 5000)  
(12, u'Mogran', 6000)  
(15, u'Jagn', 15000)  
(121, u'Mohan', 26000)  
(15, u'Jagan', 15000)  
(121, u'Mohan', 26000)  
>>>
```



The screenshot shows a database application window with a ribbon menu (Home, Create, External Data, Database Tools, Datasheet) and a table named 'mytable'. The table has three columns: 'No', 'Name', and 'Salary'. The data is as follows:

No	Name	Salary
1	HARI	15000
2	SUMI	25000
3	LATHA	17000
4	ANU	18000
5	NITHYA	16000
10	Jordan	5000
12	Mogran	6000
10	Jordan	5000
12	Mogran	6000
15	Jagn	15000
121	Mohan	26000
15	Jagan	15000
121	Mohan	26000

Program to delete the records in Ms-Access through Python

```
import pyodbc
```

```
conn = pyodbc.connect(r'Driver={Microsoft Access Driver (*.mdb, *.accdb)};DBQ=d:\Ramya_PyDB\Mydatabase.accdb;')  
cursor = conn.cursor()
```

```
data3 = input('Please enter salary: ')  
conn.execute('DELETE FROM mytable WHERE salary=?',(data3))  
conn.commit()
```

```
cursor.execute('select * from mytable')
```

```
for row in cursor.fetchall():  
    print (row)
```

Output

```
Python 2.7.15 Shell
File Edit Shell Debug Options Window Help
>>>
===== RESTART: D:/Ramya_PyDB/Data1.py =====
=====
Please enter salary: 15000
(2, u'SUMI', 25000)
(3, u'LATHA', 17000)
(4, u'ANU', 18000)
(5, u'NITHYA', 16000)
(10, u'Jordan', 5000)
(12, u'Mogran', 6000)
(10, u'Jordan', 5000)
(12, u'Mogran', 6000)
(121, u'Mohan', 26000)
(121, u'Mohan', 26000)
>>>
```

Program to Add the Records in Ms-Access Using Tkinter

```
from tkinter import *
import pyodbc
def add():
    db = pyodbc.connect(r'Driver={Microsoft Access Driver (*.mdb, *.accdb)};DBQ=d:\ramya\new.accdb;')
    cursor = db.cursor()
    #cursor.execute("insert into emp values (?)", (t2.get()))
    cursor.execute("insert into emp values (?, ?, ?, ?)", (t2.get(), c1.get(), t3.get(), t4.get()))
    db.commit()
    db.close()

group=Tk()

t2=StringVar()
t3=StringVar()
t4=StringVar()
c1=StringVar()

l1=Label(group,text="EmpName").grid(row=2,column=3)
w1=Entry(group,textvariable=t2).grid(row=2,column=20)
l2=Label(group,text="EmpDesig").grid(row=6,column=3)
w3=Entry(group,textvariable=t3).grid(row=6,column=20)
l3=Label(group,text="EmpGen").grid(row=4,column=3)
r1=Radiobutton(group,text="Male",variable=c1,value="M").grid(row=4,column=20)
r2=Radiobutton(group,text="FeMale",variable=c1,value="F").grid(row=4,column=25)
b1=Button(group,text="Add",command=add).grid(row=7,column=25)
w4=Entry(group,textvariable=t4).grid(row=8,column=20)
l4=Label(group,text="Salary").grid(row=8,column=3)
c1.set("M")

group.mainloop()
```

Output

tk

EmpName Girish

EmpGen ☒ Male ☐ FeMale

EmpDesig Manager

Salary 15000

Add

new : Database (Access 2007) - Microsoft Access

Table Tools

Home Create External Data Database Tools Datasheet

View Paste Font Rich Text Records

All Tables

emp

emp : Table

emp1

emp1 : Table

Name	Gender	Designation	Salary
Ramya	F	Faculty	15000
Deepath	M	Faculty	30000
Girish	M	Manager	15000
*			

Program to Add the Records in Ms-Access Using Tkinter

```
from tkinter import *
import pyodbc
def add():
    db = pyodbc.connect(r'Driver={Microsoft Access Driver (*.mdb, *.accdb)};DBQ=d:\ramya\new.accdb;')
    cursor = db.cursor()
    #cursor.execute("insert into emp values (?)", (t2.get()))
    cursor.execute("insert into emp values (?, ?, ?, ?)", (t2.get(), c1.get(), t3.get(), t4.get()))
    db.commit()
    db.close()

group=Tk()

t2=StringVar()
t3=StringVar()
t4=StringVar()
c1=StringVar()

l1=Label(group,text="EmpName").grid(row=2,column=3)
w1=Entry(group,textvariable=t2).grid(row=2,column=20)
l2=Label(group,text="EmpDesig").grid(row=6,column=3)
w3=Entry(group,textvariable=t3).grid(row=6,column=20)
l3=Label(group,text="EmpGen").grid(row=4,column=3)
r1=Radiobutton(group,text="Male",variable=c1,value="M").grid(row=4,column=20)
r2=Radiobutton(group,text="FeMale",variable=c1,value="F").grid(row=4,column=25)
b1=Button(group,text="Add",command=add).grid(row=7,column=25)
w4=Entry(group,textvariable=t4).grid(row=8,column=20)
l4=Label(group,text="Salary").grid(row=8,column=3)
c1.set("M")

group.mainloop()
```


Program to Add the Records in Sqlite3

```
import sqlite3
connection = sqlite3.connect("info.db")
cursor = connection.cursor()
#cursor.execute("DROP Table student")
cursor.execute("create table stu1(name, age)")
print("Enter 3 students names and their ages respectively:")
for i in range(3):
    who =[input("Enter Name:")]
    age =[int(input("Enter Age:"))]
    n =len(who)
    for i in range(n):
        cursor.execute("insert into stu1 values (?, ?)", (who[i],age[i]))
        cursor.execute("select * from stu1 order by age desc")
print("Displaying All the Records From studentmy Table in Descending
order of age")
print (*cursor.fetchall(),sep='\n' )
```

Output

```
Python 3.4.3 Shell
File Edit Shell Debug Options Window Help
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 24 2015, 22:43:06) [MSC v.1600 32
n win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Enter 3 students names and their ages respectively:
Enter Name:Ramya
Enter Age:31
Enter Name:Nandhini
Enter Age:23
Enter Name:Krithikesh
Enter Age:22
Displaying All the Records From studentmy Table in Descending order of age
('Ramya', 31)
('Nandhini', 23)
('Krithikesh', 22)
>>> |
```